



AWS Academy Cloud Developing
Module 09 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

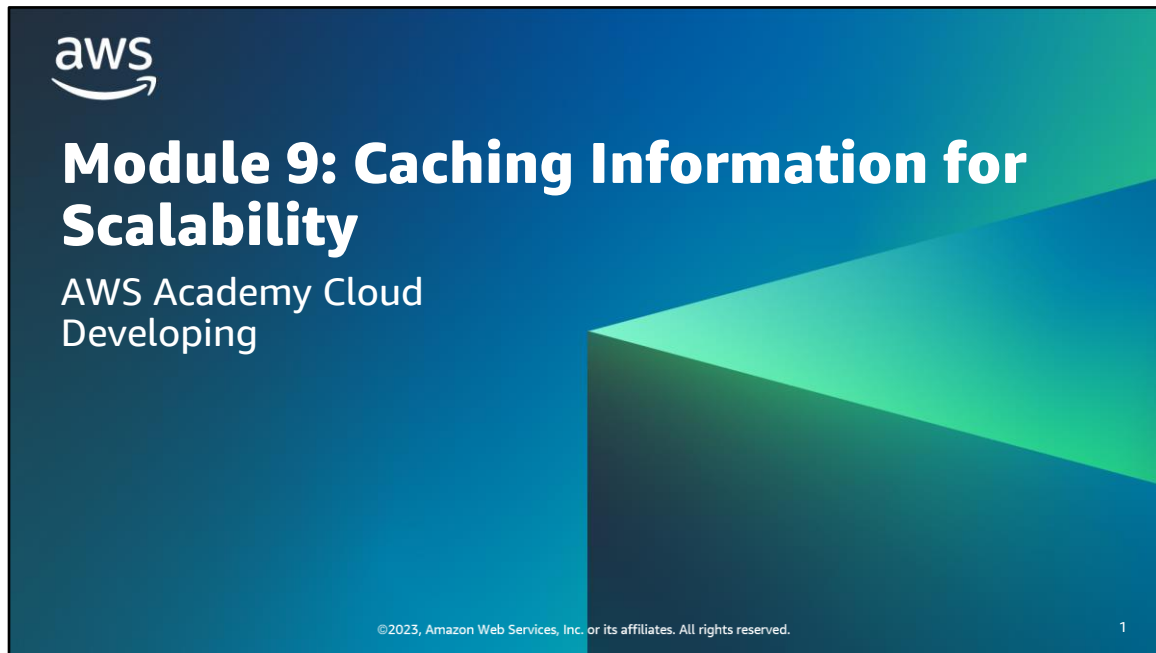
This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

All trademarks are the property of their owners.

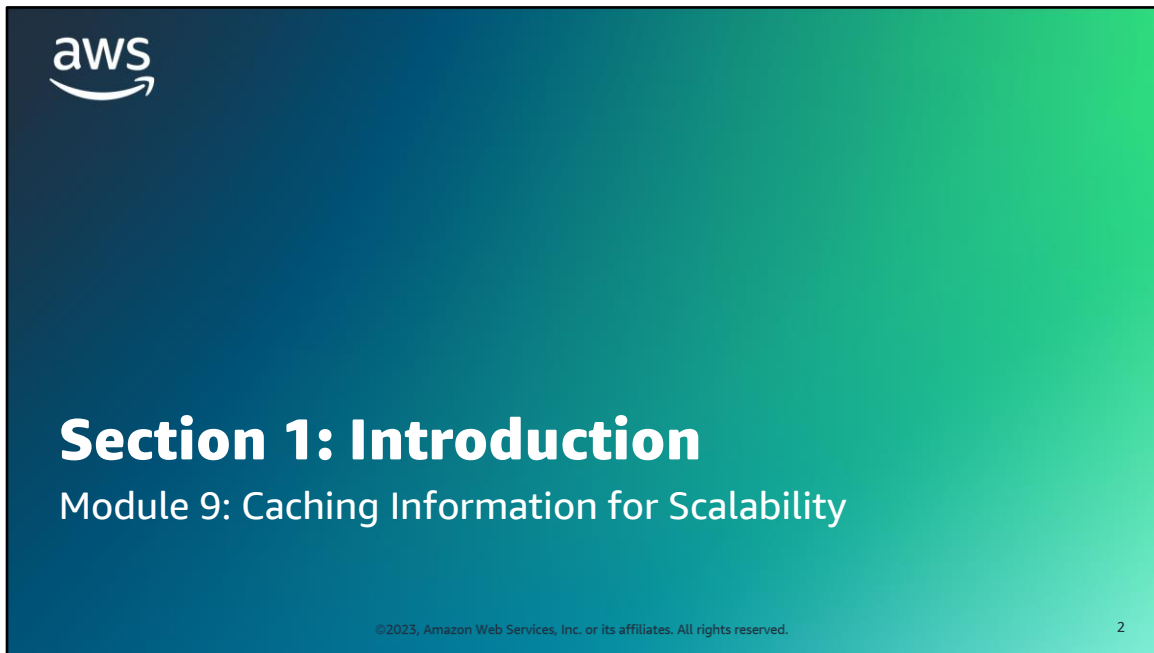
Contents

[Module 9: Caching Information for Scalability](#)

4



Welcome to Module 9: Caching Information for Scalability.



Section 1: Introduction.

Module objectives



This module prepares you to do the following:

- Explain when caching is used
- Describe caching with Amazon ElastiCache
- Describe caching with Amazon CloudFront
- Describe and compare Lambda@Edge and CloudFront Functions
- Apply caching strategies

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to do the following:

- Explain when caching is used
- Describe caching with Amazon ElastiCache
- Describe caching with Amazon CloudFront
- Describe and compare Lambda@Edge and CloudFront Functions
- Apply caching strategies

Module overview

Sections

1. Introduction
2. Caching overview
3. Caching with ElastiCache
4. Caching with CloudFront
5. Caching strategies

Labs

- Caching Application Data with ElastiCache
- Implementing CloudFront for Caching and Application Security



Knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

1. Introduction
2. Caching overview
3. Caching with ElastiCache
4. Caching with CloudFront
5. Caching strategies

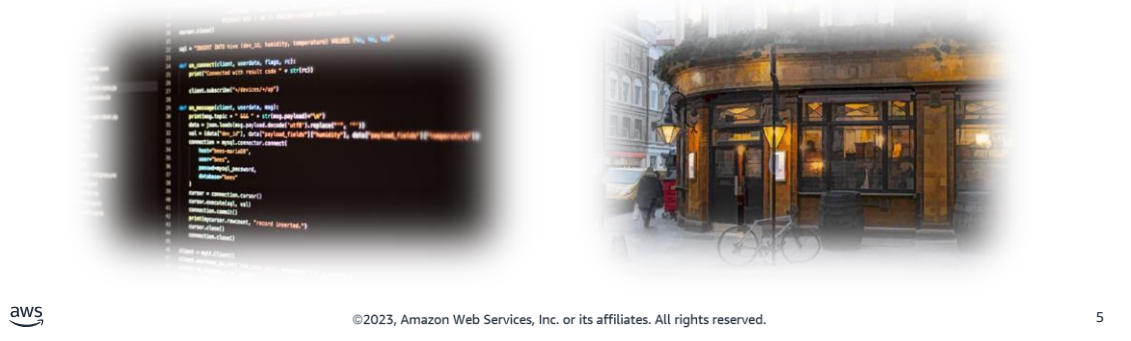
This module also includes:

- A lab where you work with Amazon ElastiCache
- A lab where you work with Amazon CloudFront

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

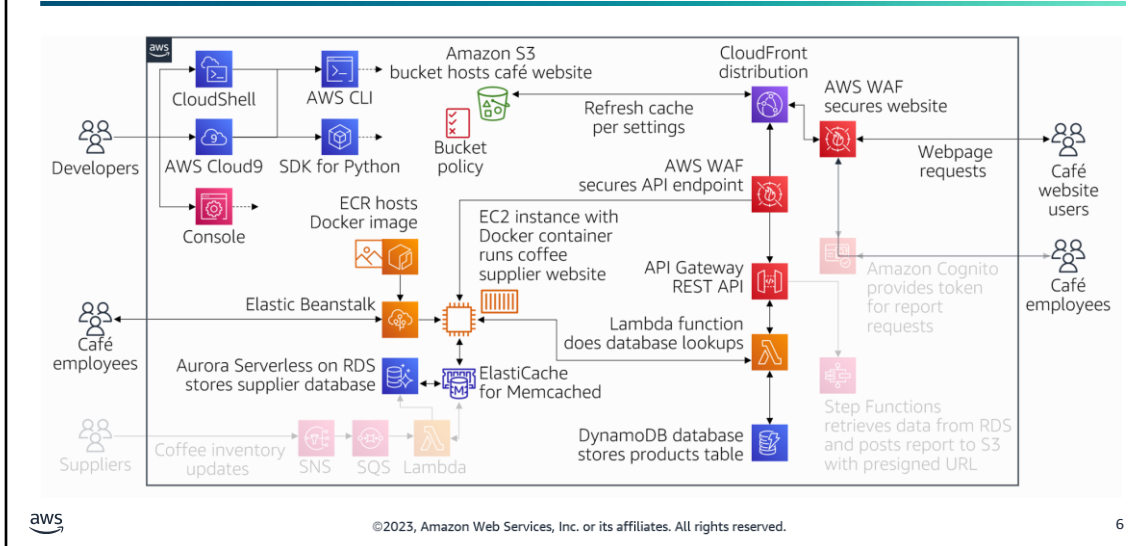
The café employees have gotten occasional customer feedback that the coffee bean inventory information takes too long to display on the café website. Sofía wants to reduce the latency and also update the site to support HTTPS traffic.



The café employees have gotten occasional customer feedback that the coffee bean inventory information takes too long to display on the café website. Sofía wants to reduce the latency and also update the site to support HTTPS traffic.

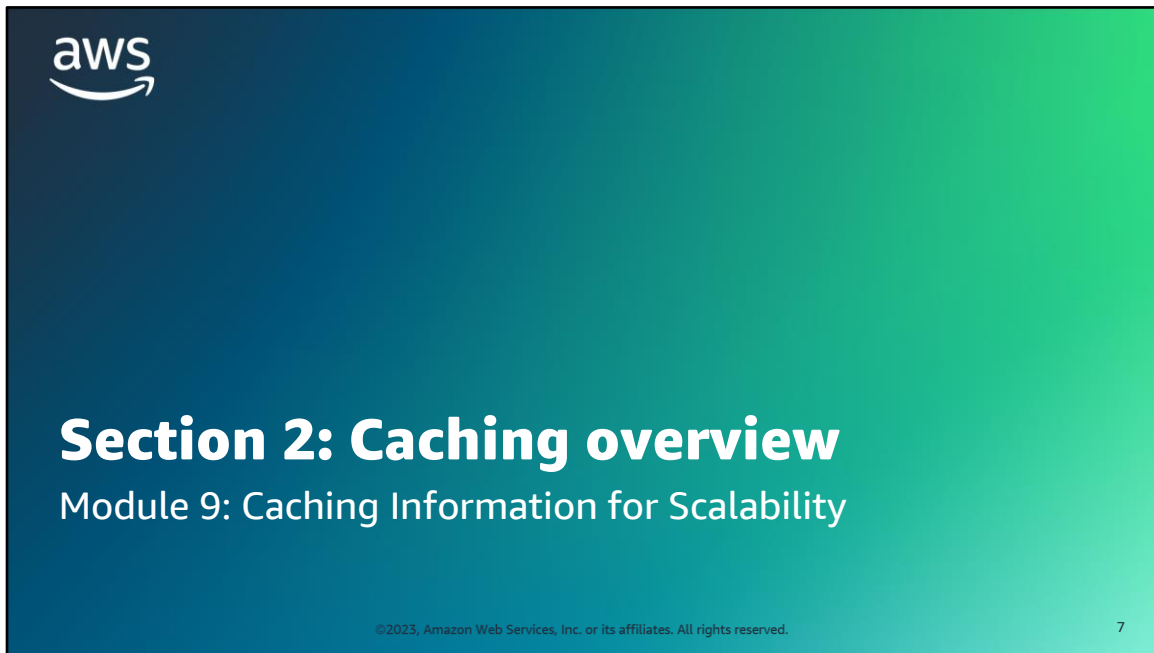
Olivia, the AWS consultant, proposed adding database caching to improve the speed of returning the coffee bean inventory information. Sofía wants to set up a content delivery network (CDN) to ensure that the site loads as quickly as possible and to support HTTPS.

Caching as part of developing a cloud application

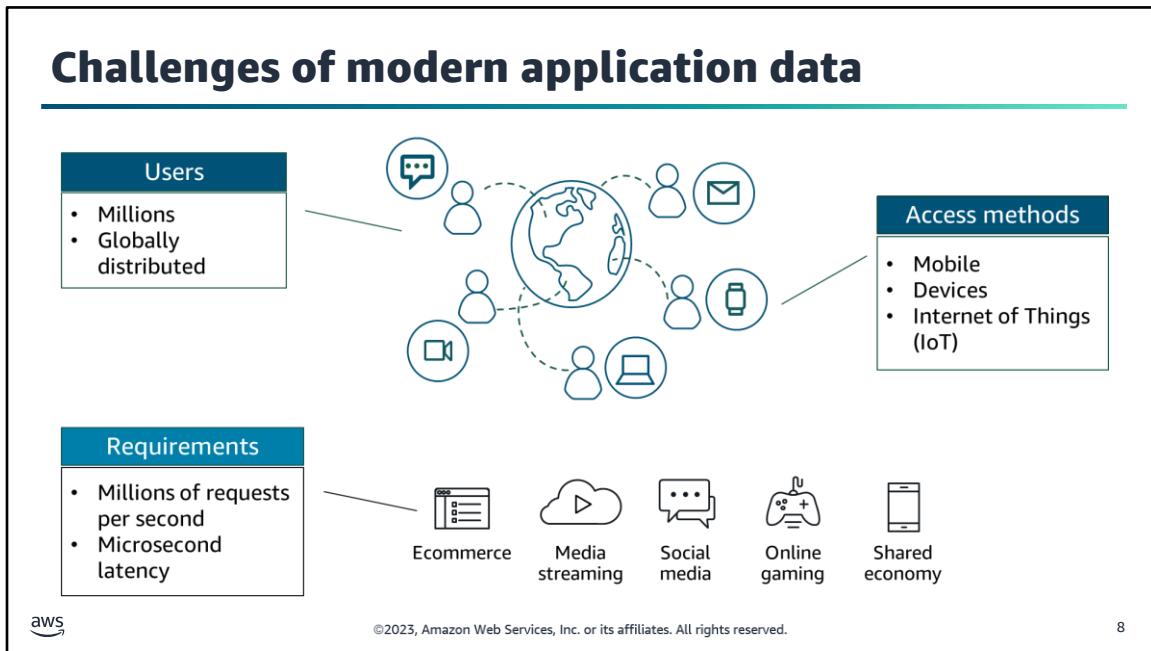


The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, in the first lab, you add database caching using Amazon ElastiCache for Memcached. In the second lab, you configure an Amazon CloudFront distribution to reduce network latency and deliver website content securely over HTTPS.



Section 2: Caching overview.

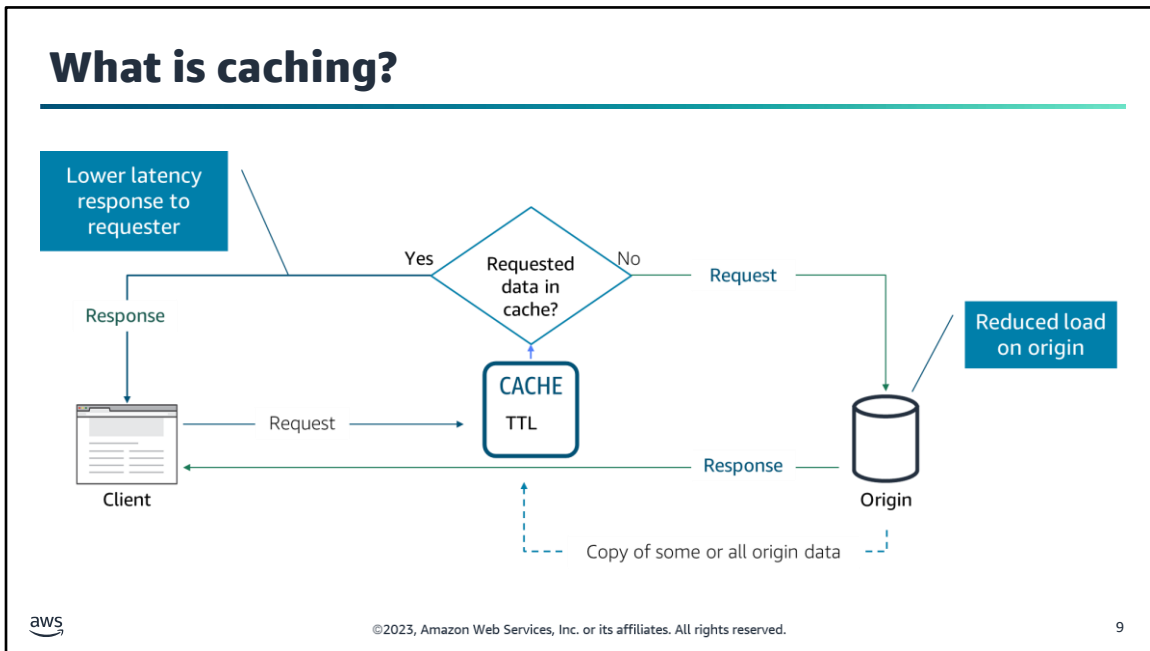


Earlier in the course, you heard something about the move from monolithic applications toward modern, microservice-based applications. Modern applications continue to evolve in scope and complexity, and the demands on the infrastructure for performance and scale go in only one direction: up.

Applications have evolved from single-server applications with hundreds of users to distributed global applications that millions use. All those users are sending requests and expecting immediate responses. Think about the data expectations for applications that use Internet of Things (IoT) devices in the field to send millions of requests. As another example, consider the data necessary for a bidding website where a few hundred microseconds would be meaningful to the outcome. Still another example is a massive global online game where users are interacting with the game and each other, which would also be data-intensive.

Developers must rethink how they store and access data to meet these performance demands, and how to support the scale and reach of modern applications.

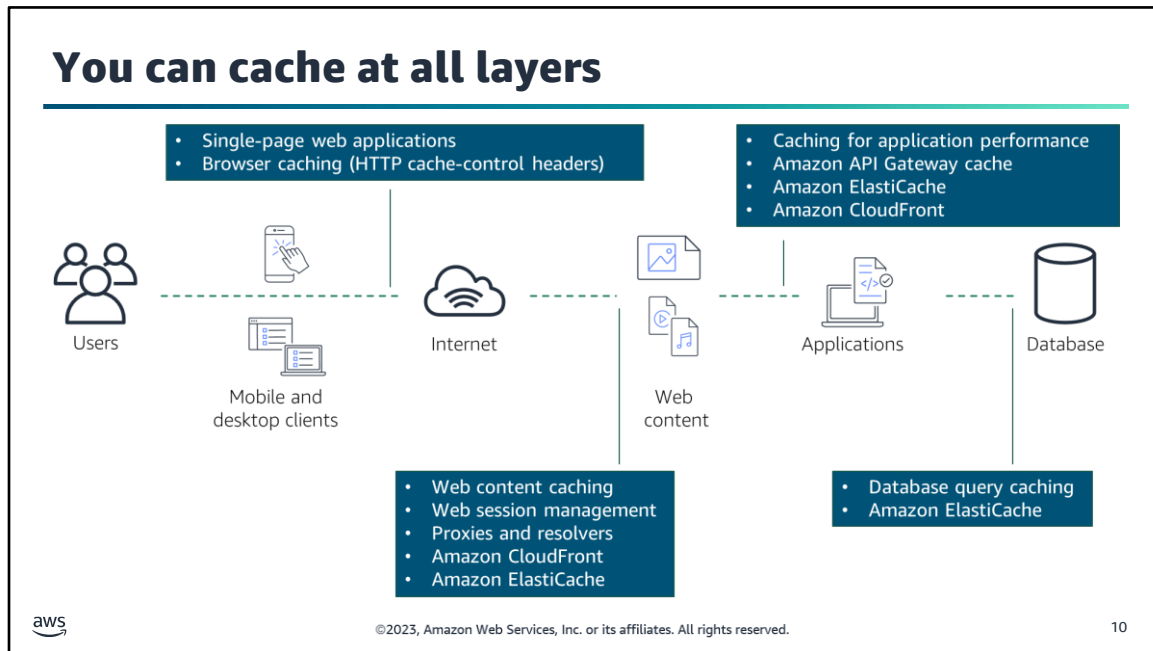
Caching is one strategy for addressing these challenges.



At the simplest level, caching is serving requests from a copy of data rather than making the request to the original source of the data. The cache is the copy of the data, and the original source of the data is called the origin. The cache is a high-speed data storage layer, and it stores data transiently. Data in the cache has a time to live (TTL) so that the data in the cache doesn't get too stale.

The primary purpose of a cache is to increase the performance of data retrieval by reducing the need to access the origin. This improvement can be in the form of faster responses back to the client, reduced load on downstream services and resources, or both.

You learned about an example of caching in the module on Amazon API Gateway. When you turn the cache on in API Gateway, API Gateway checks the cache first to see whether the requested data exists there. If it does, then API Gateway returns the response from the cache without needing to go to the backend data storage layer. This process can reduce the latency of responding to the request, and it can reduce the number of calls that reach your backend services.



You can apply caches and use them throughout various layers of technology to improve performance:

- *Accelerate retrieval of web content* on the client side by using HTTP cache-control headers.
- *Accelerate retrieval of web content from web servers and manage web sessions* on the server side. You can use Amazon CloudFront, Amazon ElastiCache for Redis, Amazon ElastiCache for Memcached, and AWS Partner solutions.
- *Accelerate application performance and data access* by using ElastiCache for Redis, ElastiCache for Memcached, and AWS Partner solutions.
- *Reduce latency that is associated with database query requests* by using ElastiCache for Redis and ElastiCache for Memcached.

In this module, you will learn about ElastiCache and CloudFront. However, you might use other caching options at different layers of your application, depending on the specific performance requirements and data that your application needs.

Workloads that benefit from caching



Read-Heavy Workloads	Compute-Intensive Workloads
Benefits • Reduce latency • Improve input/output operations per second (IOPS)	Benefits • Manipulate large datasets in real time
Examples • Social networking • Gaming • Media sharing • Question and answer (Q&A) portals	Examples • Recommendation engines • High performance computing (HPC) simulations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

Caching can benefit both read-heavy and compute-intensive workloads:

- *Read-heavy application workloads:* Caching can significantly reduce latency and improve input/output operations per second (IOPS) for many read-heavy application workloads, such as social networking, gaming, media sharing, and question and answer (Q&A) portals.
- *Compute-intensive workloads:* Caching can also benefit compute-intensive workloads that manipulate datasets, such as recommendation engines and high performance computing (HPC) simulations. In these applications, large datasets must be accessed in real-time across clusters of machines, which can span hundreds of nodes. Due to the speed of the underlying hardware, manipulating this data in a disk-based store is a significant bottleneck for these applications.

Factors for considering whether to cache



Is a **slow, expensive query** required to generate the data?



Is the data **relatively static and frequently accessed**?



Can the data afford to be **stale**?



Will caching be worth the added **development complexity and potential costs**?



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

12

Based on application requirements, developers choose strategies for how often to refresh the cache, and which origin data to cache. When deciding what data to cache, consider these factors:

- **Speed and expense:** It's generally slower and more expensive to acquire data from a database than from a cache. Some database queries are inherently slower and more expensive than others. For example, queries that perform joins on multiple tables are significantly slower and more expensive than simple queries on single tables. If the data requires a slow and expensive query to acquire data, it's a candidate for caching. If acquiring the data requires a relatively quick and simple query, it still might be a candidate for caching, depending on other factors.
- **Data and access patterns:** Determining what to cache also involves understanding the data itself and its access patterns. For example, it doesn't make sense to cache data that is rapidly changing or infrequently accessed. For caching to provide meaningful benefit, the data should be relatively static and frequently accessed, such as a personal profile on a social media site. Conversely, you don't want to cache data if caching it provides no speed or cost advantage. For example, it doesn't make sense to cache webpages that return the results of a search because these queries and results are almost always unique.
- **Staleness:** By definition, cached data is stale data. Even if it's not stale in certain circumstances, cached data should always be considered and treated as stale. When you determine whether your data is a candidate for caching, you must also determine your application's tolerance for stale data. Your application might be able to tolerate stale data in one context, but not another. For example, consider an application that serves a publicly traded stock price on a website. For this application, a short delay might be acceptable if it has a disclaimer that prices might be up to n minutes delayed. But when an application must serve the stock

price to a broker who is making a sale or purchase, you want real-time data.

- **Trade-offs:** Depending on your caching strategy, caching can add complexity to your development effort. You must weigh the benefit to your workload against the effort to write code to cache data, or potentially handle security for an additional vector of attack. Caching can cause additional costs, so optimizing for cost is an important part of deciding what and how to cache. As a developer, you must understand how caching might benefit your application, but you want to be wary of optimizing too early. It is best to wait until the application is complete so that you can see where the bottlenecks are. Next, monitor access patterns and performance in production-like scenarios, and then apply caching where it adds the most benefit.

Section 2 key takeaways



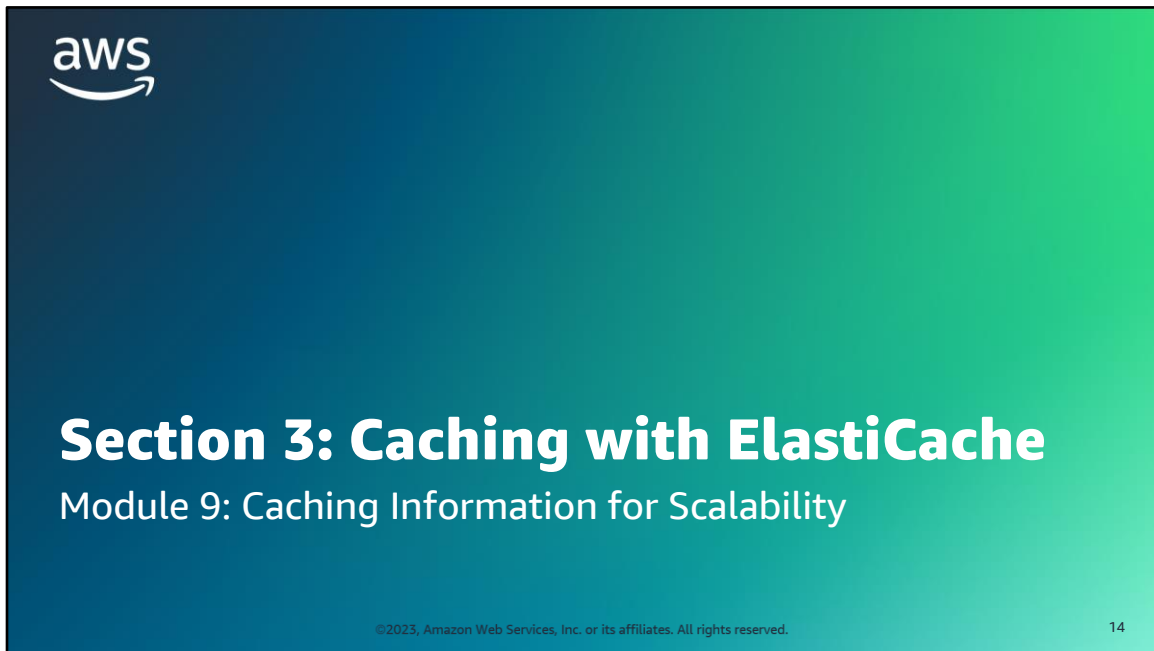
- Modern applications put high demands on the **performance of data sources** in an application.
- Caching can improve performance by serving requests from a **copy of data** (a cache) rather than making the request to the origin.
- Caching **can occur at all layers** of your application.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

13

The following are the key takeaways from this section of the module:

- Modern applications put high demands on the performance of data sources in an application.
- Caching can improve performance by serving requests from a copy of data (a cache) rather than making the request to the origin.
- Caching can occur at all layers of your application.



Section 3: Caching with ElastiCache.

ElastiCache



Amazon
ElastiCache



Fully managed **in-memory** data store with **sub-millisecond** latency

- Key-value, in-memory store that sits between your application and the origin data store
- Decreases access latency, increases throughput, and eases load of databases and applications
- Scales out, in, up, or down to meet demands
- Fully compatible with open-source Redis and Memcached engines

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

15

Amazon ElastiCache is an *in-memory, key-value store* that sits between your application and the data store (database) that it accesses. The primary purpose of an in-memory, key-value store is to provide ultrafast (sub-millisecond latency) and inexpensive access to copies of data.

Most data stores have areas of data that are frequently accessed but seldom updated. Additionally, querying a database is always slower and more expensive than locating a key in a key-value pair cache. Some database queries are especially expensive to perform, such as queries that involve joins across multiple tables or queries with intensive calculations. By caching these kinds of query results, you pay the price of the query one time. Then, you can quickly retrieve the data multiple times without having to rerun the query.

The benefits of ElastiCache include the following:

- **Fully managed:** ElastiCache manages the work involved in setting up a distributed in-memory environment, from provisioning the server resources that you request to installing the software. After your environment is up and running, the service automates common administrative tasks, such as failure detection, recovery, and software patching. ElastiCache provides detailed monitoring metrics that are associated with your nodes, enabling you to diagnose and react to issues quickly. For example, you can set up thresholds and receive alarms if one of your nodes is overloaded with requests.
- **Extreme performance:** ElastiCache works as an in-memory data store and cache to support the most demanding applications that require sub-millisecond response times. By using an end-to-end, optimized stack that runs on customer-dedicated nodes, ElastiCache provides secure, fast performance.
- **Scalable:** ElastiCache can scale out, scale in, and scale up to meet fluctuating application demands.

ElastiCache engines



ElastiCache for Memcached	ElastiCache for Redis
Features • Multithreading • Low maintenance • Easy horizontal scalability with Auto Discovery	Features • Support for complex data types • Persistence • High availability (replication) • Automatic failover • Publish/subscribe capabilities
Suitability • Need the simplest model possible	Suitability • Need the advantage of data persistence • Need advanced data types • Need one of the other features that are listed



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

16

ElastiCache supports two open-source, in-memory caching engines: Memcached and Redis.

ElastiCache for Memcached offers the following:

- **Multithreading:** With this feature, Memcached can use more CPU cores than Redis.
- **Low maintenance:** Memcached is designed to be relatively simple, and so it requires less maintenance than Redis.
- **Easy horizontal scalability with Auto Discovery:** When Memcached clusters are used in ElastiCache, the clusters can easily add and remove nodes by using the Auto Discovery feature. This feature automatically discovers changes to the node membership in a cluster, such as new nodes that are added or nodes that were subtracted. The ElastiCache client reconfigures automatically in response.

ElastiCache for Redis offers the following:

- **Support for complex data types:** Examples of these data types include strings, hashes, lists, sets, sorted sets, and bitmaps. By contrast, Memcached is designed to cache flat strings—such as flat HTML pages or serialized JSON—and objects, like databases.
- **Persistence:** Data in Redis has persistence, so you can actually use it as a primary data store. The Memcached engine does not support persistence. Thus, if a new empty node replaces a failed node, or you terminate or scale down a node, you lose the data in cache memory.
- **High availability (replication):** Replicate your data from the primary node to one or more read replicas for read-intensive applications.
- **Automatic failover:** In certain cases (for example, planned maintenance, or failure of primary node or Availability Zone), Redis detects and replaces the primary node. If you enabled Multi-AZ deployment with automatic failover on the cluster, the role

of the primary node fails over to one of the read replicas.

- **Publish/subscribe capabilities:** When you send an email message, you send it to one or more specified recipients. In the publish/subscribe paradigm, you send a message to a specific channel without knowing who, if anyone, receives it. Subscribers to the channel receive the message. You can use Redis to manage the channels and enable quick messaging and communication between publishers and subscribers.

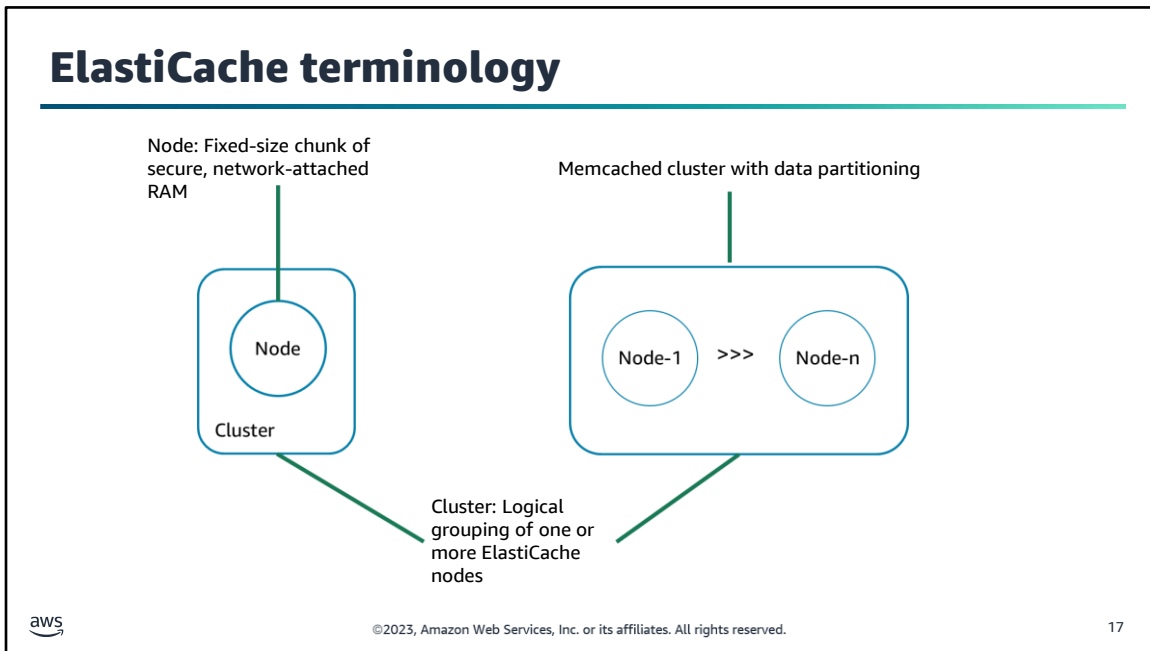
Overall, if you must decide which ElastiCache engine to use, base your decision on a use case that can justify the use of Redis. Memcached is typically preferred when your cache doesn't need the advanced features that Redis offers.

Choose Memcached if you need the simplest model possible.

However, if you need the advantages of data persistence, advanced data types, or the other features that are listed, Redis is probably your best solution.

For more information, see the following resources:

- Comparing Redis and Memcached: <https://aws.amazon.com/elasticache/redis-vs-memcached/>.
- Comparing Memcached and Redis: <https://docs.aws.amazon.com/AmazonElastiCache/latest/mem-ug/SelectEngine.html>.



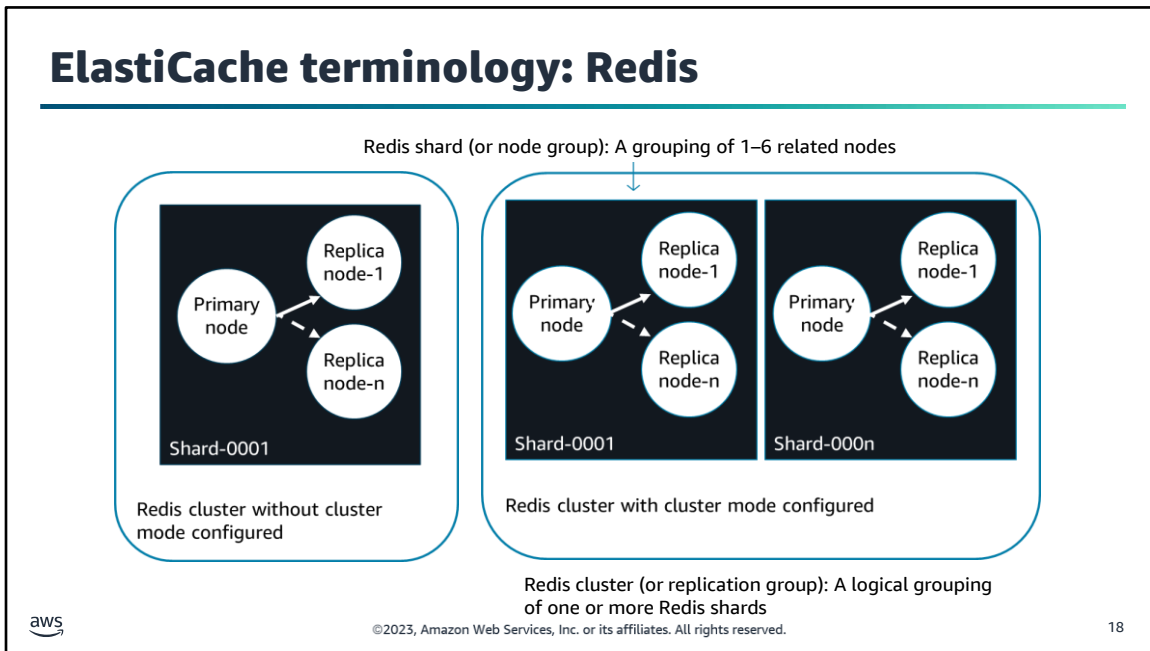
A node is a fixed-size chunk of secure, network-attached RAM. A cluster is a logical grouping of one or more ElastiCache nodes. Each node runs an instance of the engine and version that was chosen when you created your cluster. If necessary, you can scale the nodes in a cluster up or down to a different instance type.

Every node within a cluster is the same instance type and runs the same cache engine. Each cache node has its own DNS name and port. Multiple types of cache nodes are supported, each with varying amounts of associated memory.

Your application connects to an ElastiCache node or cluster by using a unique address that is called an endpoint.

With the Memcached engine, data is partitioned across nodes in a cluster. Memcached data is partitioned across up to 20 nodes. Memcached does not have replication.

For more information about ElastiCache for Memcached components and features, see the Amazon ElastiCache for Memcached User Guide at <https://docs.aws.amazon.com/AmazonElastiCache/latest/mem-ug/WhatIs.Components.html>.



The Redis engine has additional components. As new functionality was introduced into newer versions of ElastiCache for Redis to support new Redis features, some component names were modified. The console reflects the updated terminology, but to preserve backward compatibility, the API and command line interface (CLI) use the earlier terminology.

A *Redis shard* (which is called a node group in the API and CLI) is a grouping of one to six related nodes. If a shard has more than one node, the shard supports replication. In this case, one node is the read/write primary node, and the others are read-only replica nodes.

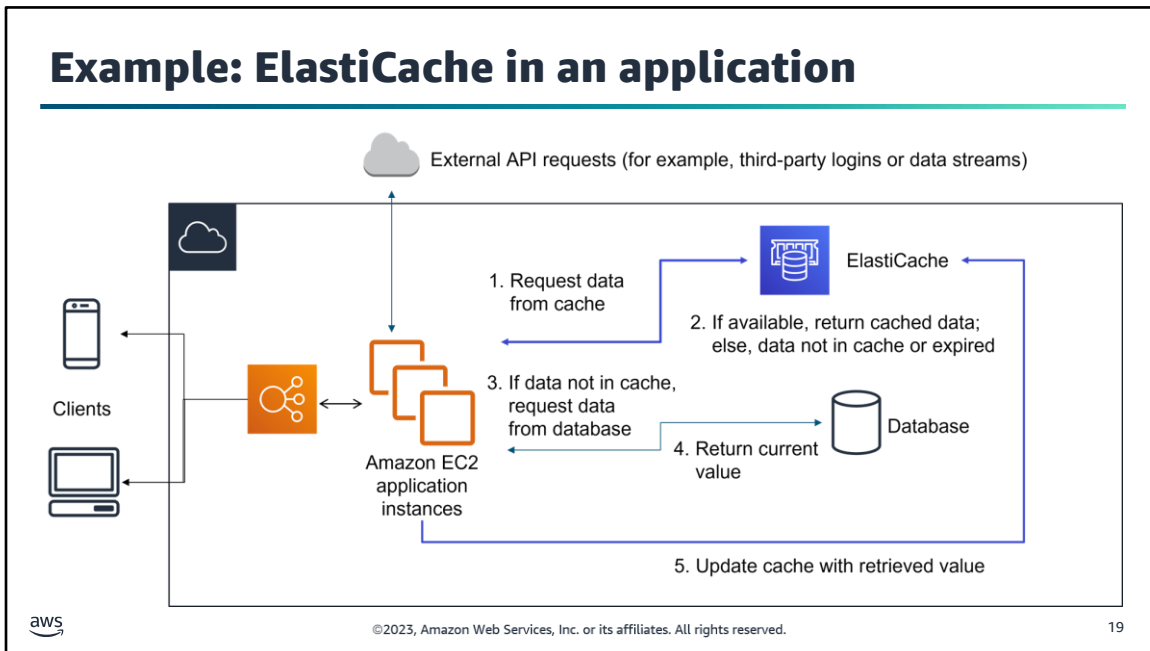
A *Redis cluster* (which is called a replication group in the API and CLI) is a logical grouping of one or more shards.

- A Redis cluster that is configured without cluster mode always has one shard, and this configuration has no data partitioning.
- A Redis cluster that is configured with cluster mode can have up to 500 shards, and your data is partitioned across the shards.

For more information about ElastiCache for Redis, see the following resources:

- ElastiCache for Redis Terminology:
<https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/WhatIs.Terms.html>.
- ElastiCache for Redis Components and Features:
<https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/WhatIs.Components.html>.

For more information about supported node types, see the Amazon ElastiCache for Redis User Guide at <https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/CacheNodes.SupportedTypes.html>.



Amazon ElastiCache makes it easy to deploy, operate, and scale an in-memory data store or cache in the cloud. You can use ElastiCache to improve the performance of your web applications. With ElastiCache, you can retrieve information from fast, managed, in-memory data stores, instead of relying entirely on slower disk-based databases.

As a developer, you don't have to worry about actively managing ElastiCache after your cluster is up and running. ElastiCache is a managed service, so it handles administrative tasks like provisioning, failure detection, and software patching on your behalf.

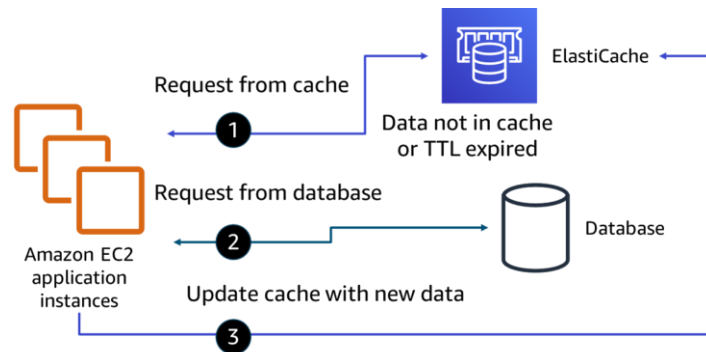
ElastiCache integrates with other AWS services, including Amazon Elastic Compute Cloud (Amazon EC2), Amazon Relational Database Service (Amazon RDS), and Amazon DynamoDB. ElastiCache also integrates with deployment management solutions, such as AWS CloudFormation, AWS Elastic Beanstalk, and AWS OpsWorks. Therefore, it is easier for you to focus on the application logic and not the infrastructure.

Your focus as a developer is on writing the code to handle these logical actions:

1. Request data from the cache:
 - If you find the data in the cache and it has not expired (which is called a *cache hit*), go to step 2.
 - If you don't find the data you need in the cache or the cached value has expired (which is called a *cache miss*), go to step 3.
2. Return it to the requester.
3. Request the data from the origin.
4. Return the data to the requester.
5. Update the cache with the data that is retrieved from the origin.

In this example architecture, Elastic Load Balancing distributes application traffic across multiple EC2 application instances, which are making external API requests. Numerous database reading requests for the same data increases the response time, so you can add ElastiCache to improve response times and alleviate database pressure. The application reads from the cache. If the information is not available in the cache or is expired, then the application reads from the database. (This database might be an Amazon RDS DB instance, DynamoDB, or another database.) If the database receives numerous read requests for the same data, the application writes the data that is received from the store to the cache. In this way, the data can be retrieved more quickly the next time that it is requested.

Cache miss penalty and time to live (TTL)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

20

Notice that in the example, with a cache miss, three trips are related to the request. You must send the initial request to the cache, then query the database, and then update the cache. A cache miss can noticeably delay returning data to the application.

One strategy is to load all data into the cache in advance, rather than a subset. You might use this strategy if your application cannot tolerate the cache miss penalty for any requests.

Recall that a cache miss might happen if the desired value is not in the cache or the requested key has expired (TTL has expired).

In addition to deciding what to cache, you must decide the time to live for data in the cache. A short TTL might result in more cache misses, but extending the TTL could mean that you are returning stale data to your application.

Cache expiration can get complex quickly. In a real application, a given page or screen often caches many different kinds of data at the same time. For example, these types of data can include profile data, top news stories, recommendations, comments—all of which have different update methods.

One strategy is to always apply a TTL to all of your cache keys. TTL is an integer value that specifies the number of seconds until the key expires.

For more information about strategies for resolving cache expiration, see the Performance at Scale with Amazon ElastiCache whitepaper at <https://d1.awsstatic.com/whitepapers/performance-at-scale-with-amazon-elasticache.pdf>.

Test with production-like data, and continue to monitor caching behavior and application performance in production. This strategy optimizes your choices for what to cache and how frequently it must be refreshed.

The section on caching strategies presents additional considerations about how to use your cache.

Section 3 key takeaways



- ElastiCache is a fully managed, **in-memory** data store with **sub-millisecond** latency.
- ElastiCache supports two open-source in-memory caching engines: **Memcached** and **Redis**.
- A **node** is a fixed-size chunk of secure, network-attached RAM, and a **cluster** is a logical grouping of one or more ElastiCache nodes.

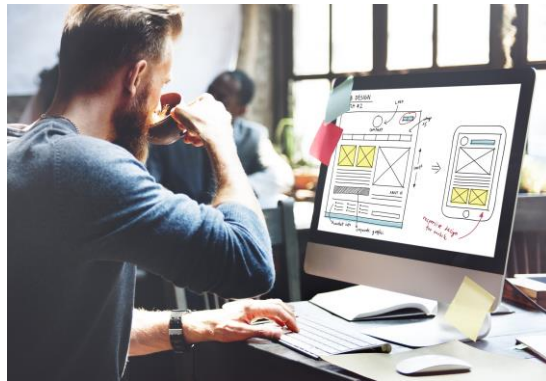
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

21

The following are the key takeaways from this section of the module:

- ElastiCache is a fully managed, in-memory data store with sub-millisecond latency.
- ElastiCache supports two open-source in-memory caching engines: Memcached and Redis.
- A node is a fixed-size chunk of secure, network-attached RAM, and a cluster is a logical grouping of one or more ElastiCache nodes.

Lab 9.1: Caching Application Data with ElastiCache



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

22

You will now complete Lab 9.1: Caching Application Data with ElastiCache.

Lab: Scenario

- Frank and Martha are excited that the bean inventory from the coffee suppliers application is integrated into the café website. However, they are getting negative feedback that the bean inventory information takes too long to display on the page.
- One of the café's customers, Olivia, who is an AWS consultant, proposed adding database caching to improve the speed of the database query.
- Olivia recommended using the ElastiCache managed service. ElastiCache is similar to Amazon Aurora Serverless in that it simplifies the process to deploy and maintain a cache cluster.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

The scenario for this lab reflects the following café decisions:

- Frank and Martha are excited that the bean inventory from the coffee suppliers application is integrated into the café website. However, they are getting negative feedback that the bean inventory information takes too long to display on the page.
- One of the café's customers, Olivia, who is an AWS consultant, proposed adding database caching to improve the speed of the database query.
- Olivia recommended using the ElastiCache managed service. ElastiCache is similar to Amazon Aurora Serverless in that it simplifies the process to deploy and maintain a cache cluster.

Lab: Tasks

1. Preparing the development environment
2. Configuring the subnets for ElastiCache to use
3. Creating the ElastiCache cluster
4. Loading records into the cache from the database
5. Updating data
6. Creating a new record
7. Deleting a record
8. Updating the application container



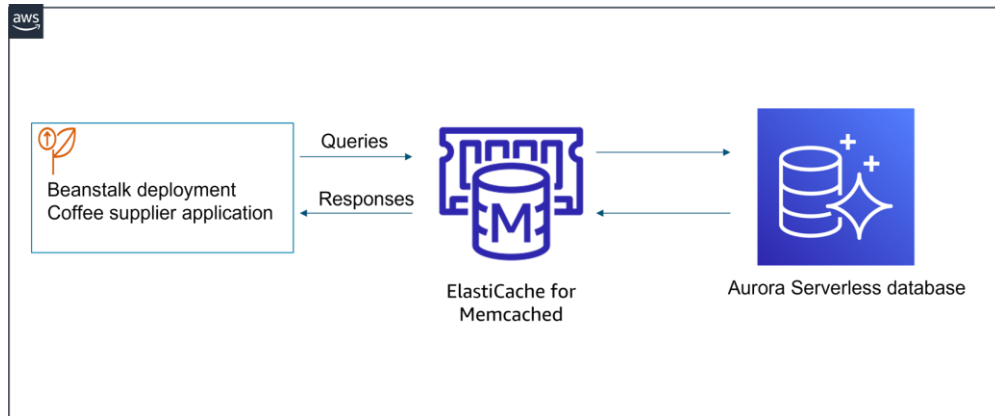
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Configuring the subnets for ElastiCache to use
3. Creating the ElastiCache cluster
4. Loading records into the cache from the database
5. Updating data
6. Creating a new record
7. Deleting a record
8. Updating the application container

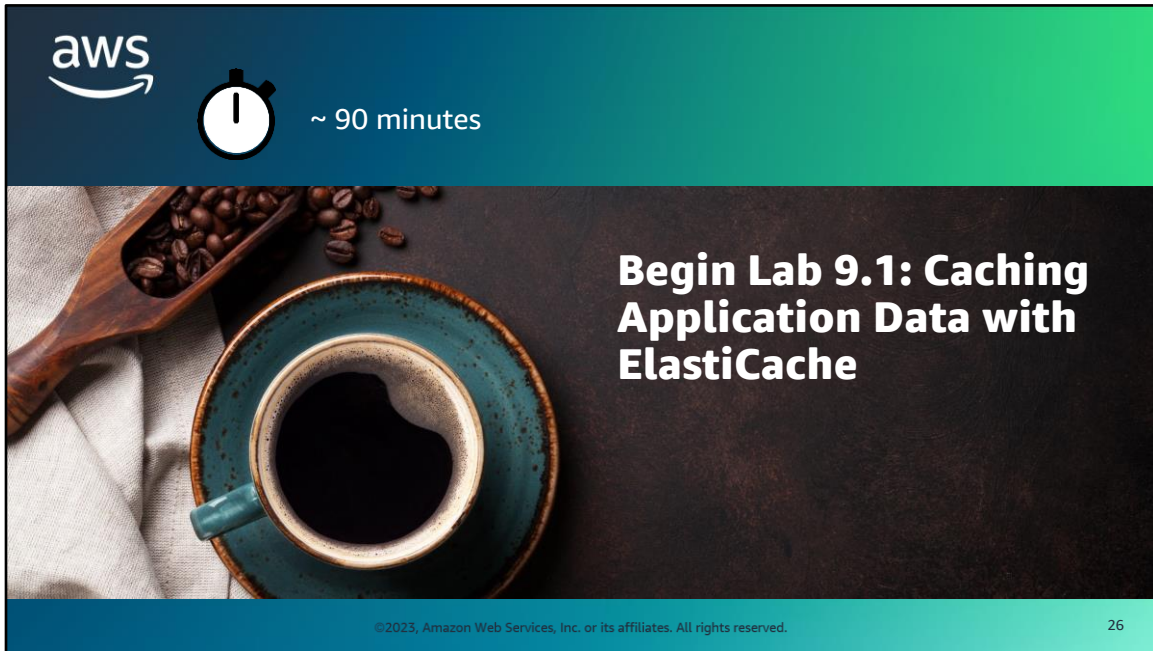
Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

25

The diagram summarizes what you will have built after you complete the lab.



The image is a slide for an AWS lab. At the top, there is a dark blue header with the AWS logo on the left, a stopwatch icon in the center, and the text '~ 90 minutes' on the right. Below the header is a large image of a cup of coffee on a saucer, with coffee beans scattered around it. To the right of the coffee image, the text 'Begin Lab 9.1: Caching Application Data with ElastiCache' is displayed in white. At the bottom of the slide, there is a teal bar containing the copyright text '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the number '26' on the right.

aws ~ 90 minutes

Begin Lab 9.1: Caching Application Data with ElastiCache

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 26

It is now time to start the lab.

Lab debrief: Key takeaways

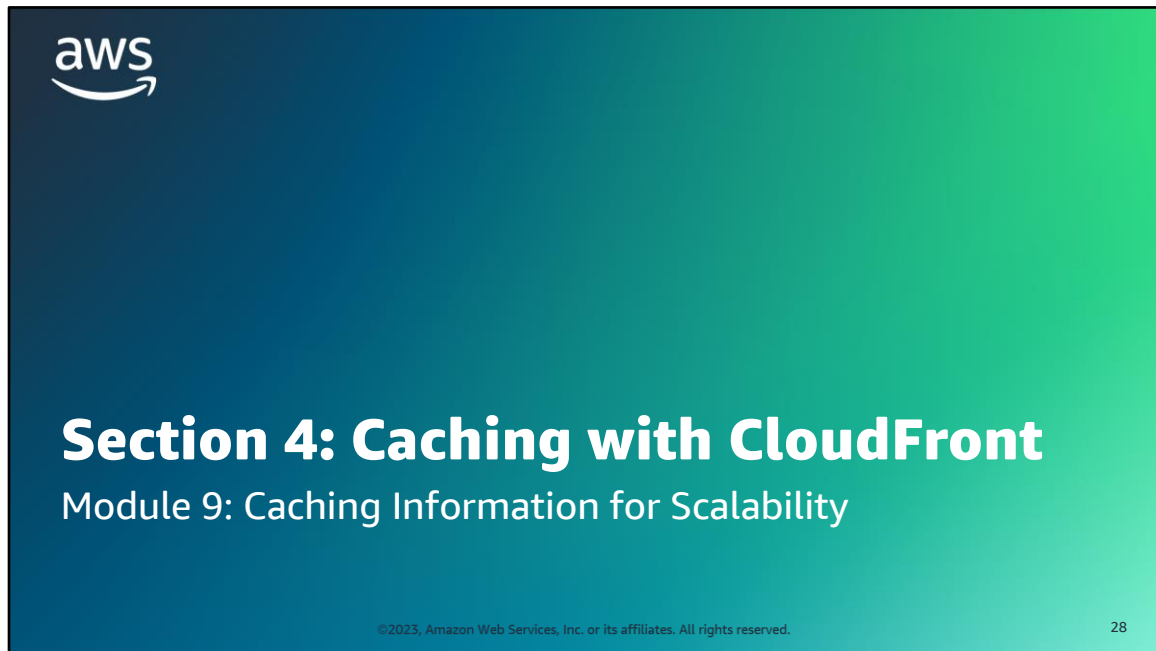


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

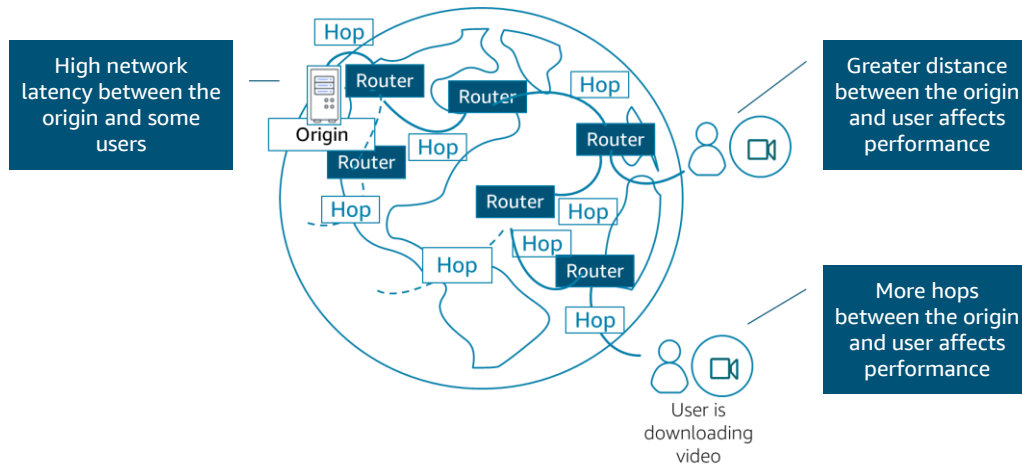
27

After you complete the lab, your educator might choose to lead a conversation about the key takeaways.



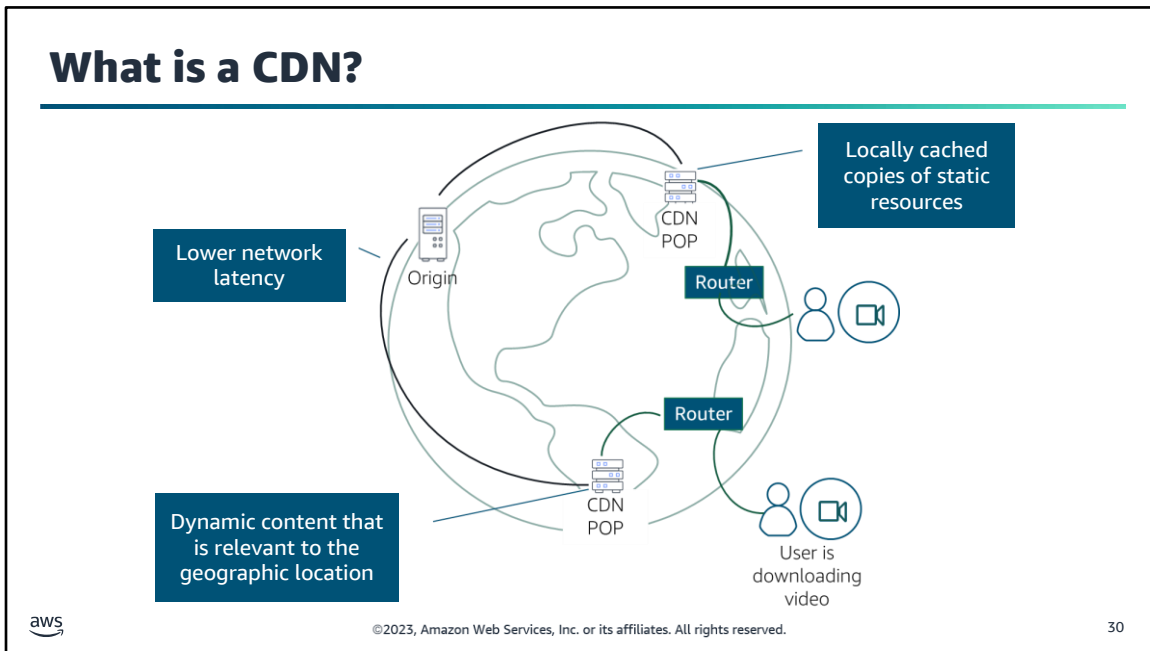
Section 4: Caching with CloudFront.

Challenges of serving data globally



When someone browses your website or uses your application, the request is routed through many different networks to reach your origin server. The origin server (or origin) stores the original, definitive versions of your objects (for example, webpages, images, or media files).

The number of network hops and the distance that the request must travel significantly affect the performance and responsiveness of your website. Further, network latency is different depending on geographic location, which is where a content delivery network (CDN) comes in.



A content delivery network (CDN) is a globally distributed system of caching servers. A CDN caches copies of commonly requested files (static content, such as HTML, CSS, JavaScript, and image files) that the application origin server is hosting. The CDN delivers a local copy of the requested content from a cache edge or Point of Presence (POP). The CDN uses the copy that provides the fastest delivery to the requester.

CDNs also deliver dynamic content that is unique to the requester and not cacheable. Having a CDN deliver dynamic content improves application performance and scaling. The CDN establishes and maintains secure connections closer to the requester. If the CDN is on the same network as the origin, routing back to the origin to retrieve dynamic content is accelerated. Furthermore, content such as form data, images, and text can be ingested and sent back to the origin. This process takes advantage of the low-latency connections and proxy behavior of the POP.

CloudFront



Amazon
CloudFront

Fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers **globally** with **low latency** and **high transfer speeds**

- Optimized for performance and scale
- Built-in security features
- Deeply integrated with AWS services
- Highly customizable



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

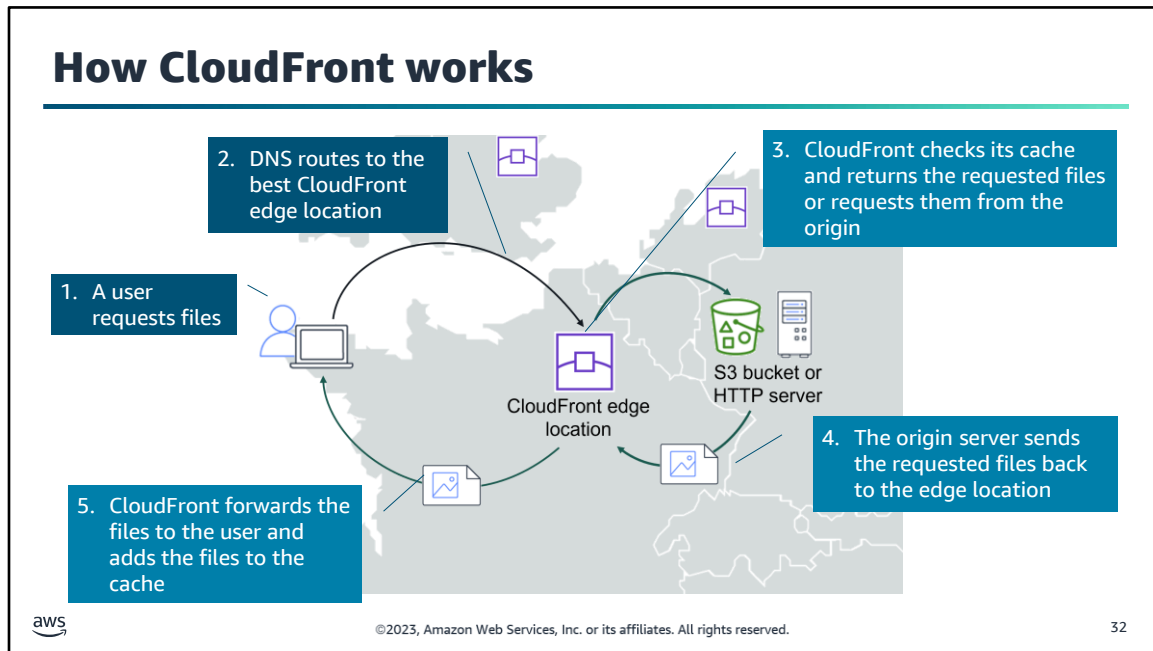
Amazon CloudFront is a CDN that securely delivers data, videos, applications, and APIs to customers globally with low latency and at high transfer speeds. These attributes all exist within a developer-friendly environment.

CloudFront integrates with AWS Shield to provide built-in security against distributed denial of service (DDoS) attacks.

It integrates with Amazon Simple Storage Service (Amazon S3), ELB, and Amazon EC2. You can use these services as AWS origins for your applications.

CloudFront is also highly customizable. It integrates with Lambda@Edge to run custom code closer to your users and to customize the user experience.

For more information about Amazon CloudFront, see <https://aws.amazon.com/cloudfront/>.



CloudFront delivers your content through a worldwide network of data centers, which are called *edge locations*. CloudFront edge locations are connected to the AWS Regions through the AWS network backbone. When a user requests content that you are serving with CloudFront, they are routed to the edge location that provides the lowest latency (or time delay). In this way, the content is delivered with the best possible performance. CloudFront edge locations ensure that popular content can be served quickly to your viewers.

The cache key is the unique identifier for every object in the cache, and it determines whether a viewer request results in a *cache hit*. A cache hit occurs when a viewer request generates the same cache key as a prior request, and the object for that cache key is in the edge location's cache and valid. When a cache hit occurs, the object is served to the viewer from a CloudFront edge location.

After you configure CloudFront to deliver your content, the following happens when users request your files:

1. A user accesses your website or application and requests one or more files, such as an image file and an HTML file.
2. DNS routes the request to the CloudFront POP (edge location) that can best serve the request, typically the nearest CloudFront POP with lowest latency.

When it receives the request, CloudFront does the following:

3. In the POP, CloudFront checks its cache for the requested files. If the files are in the cache, CloudFront returns them to the user.
4. If the files are *not* in the cache, CloudFront compares the request with the specifications in your distribution and forwards the request to your origin server for the corresponding file type. For example, it would forward the request to your S3 bucket for image files and to your HTTP server for HTML files. The origin servers send the files back to the edge location.
5. As soon as the first byte arrives from the origin, CloudFront begins to forward the files to the user. CloudFront also adds the files to the cache in the edge location.

for the next time that someone requests those files.

CloudFront also has *Regional edge caches*, which bring more of your content closer to your viewers. Regional edge caches include content even when it is not popular enough to stay at a POP, to help improve performance for that content.

Regional edge caches help with all types of content, particularly content that tends to become less popular over time. Examples include user-generated content, such as video, photos, or artwork, and ecommerce assets such as product photos and videos. Other examples might be news and event-related content that might suddenly find new popularity.

CloudFront distribution types



Web content

- Used for serving content over HTTP or HTTPS, such as static and dynamic content.
- The origin can be an HTTP server or an S3 bucket.



Video on demand

- Used for video content that users can watch at any time.
- Videos must be encoded to support streaming.
- The origin can be an HTTP server or an S3 bucket.



Live event

- Used to stream live events.
- Videos must be encoded and compressed for viewing devices.
- The origin can be AWS Elemental MediaStore or AWS Elemental MediaPackage.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

33

When using CloudFront to distribute your content, you create a *distribution* to tell CloudFront where you want content to be delivered from. This information tells CloudFront which origin servers to get your files from, and the details about how to track and manage content delivery. Then CloudFront uses edge servers that are close to your viewers to deliver that content quickly when someone wants to see it or use it.

You can use distributions to serve the following content over HTTP or HTTPS:

- Static and dynamic download content, such as .html, .css, .js, and image files.
- Video on demand in different formats, such as Apple HTTP Live Streaming (HLS) and Microsoft Smooth Streaming.
- A live event, such as a meeting, conference, or concert, in real time. For live streaming, you can create the distribution automatically by using an AWS CloudFormation stack.

You must choose the distribution type that matches your content type when creating your distribution.

CloudFront assigns a domain name to your new distribution and sends your distribution's configuration to all of its edge locations.

For more information, see the following resources:

- Overview of Distributions:
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/distribution-overview.html>.
- Video on Demand and Live Streaming Video with CloudFront:
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/distribution-overview-rtmp.html>.

Understanding the cache key

The default cache key is the CloudFront distribution domain name plus the URL path of the requested object. Other values from the request are not included in the cache key by default. Examples follow.

Cache key:

- Domain: `d111111abcdef8.cloudfront.net`
- URL: `/content/stories/example-story.html`

Request 1: No matching cache key in cache. Cache miss

```
GET /content/stories/example-story.html?ref=0123abc&split-pages=false HTTP/1.1
Host: d111111abcdef8.cloudfront.net
User-Agent: Mozilla/5.0 Gecko/20100101
Firefox/68.0
Accept: text/html,*/*
Accept-Language: en-US,en
Cookie: session_id=01234abcd
Referer: https://news.example.com/
```

Request 2: Cache key matches request 1. Cache hit

```
GET /content/stories/example-story.html?ref=xyz987&split-pages=true HTTP/1.1
Host: d111111abcdef8.cloudfront.net
User-Agent: Mozilla/5.0 AppleWebKit/537.36
Chrome/83.0.4103.116
Accept: text/html,*/*
Accept-Language: en-US,en
Cookie: session_id=wxyz9876
Referer: https://rss.news.example.net/
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

34

With CloudFront, you can control the *cache key* for objects that are cached at CloudFront edge locations.

Recall that the cache key is the unique identifier for every object in the cache. It determines whether a viewer request results in a *cache hit*.

You can get better performance from your website or application when you have a higher *cache hit ratio*. (A higher cache hit ratio means that a higher proportion of viewer requests result in a cache hit.) One way to improve your cache hit ratio is to include only the minimum necessary values in the cache key.

By default, the cache key for a CloudFront distribution includes the following information:

- The domain name of the CloudFront distribution (for example, `d111111abcdef8.cloudfront.net`)
- The URL path of the requested object (for example, `/content/stories/example-story.html`)

In this example, request 2 is for the same object as the previous request. However, the request has a different URL query string, different User-Agent and Referer headers, and a different session_id cookie. But because none of these values are part of the cache key by default, this second request results in a cache hit.

You can modify the cache key with cache policies. However, before modifying the cache key, it's important to understand how your application is designed. You want to know when and how it might serve different responses based on characteristics of the viewer request. When a value in the viewer request determines the response that your origin returns, you should include that value in the cache key. But if you include a value in the cache key that doesn't affect the response that your origin returns, you might cache duplicate objects.

For more information about improving cache hit ratios, see the Amazon CloudFront Developer Guide at <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cache-hit-ratio.html>.

Controlling cache keys with cache policies

Settings Category	Description	Available Settings
Policy information	Describes the policy.	- Name of policy (which is used to attach the policy to a caching behavior) - Comment
Time to live (TTL) settings	Work together with the Cache-Control and Expires HTTP headers (if they're in the origin response). They determine how long objects in the CloudFront cache remain valid before CloudFront checks the origin for updates.	- Minimum TTL - Maximum TTL - Default TTL
Cache key settings	Specify the values in viewer requests that CloudFront includes in the cache key. The values that you include in the cache key are automatically included in requests that CloudFront sends to the origin, which are called origin requests.	- URL query strings - HTTP headers - Cookies - Compression support



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

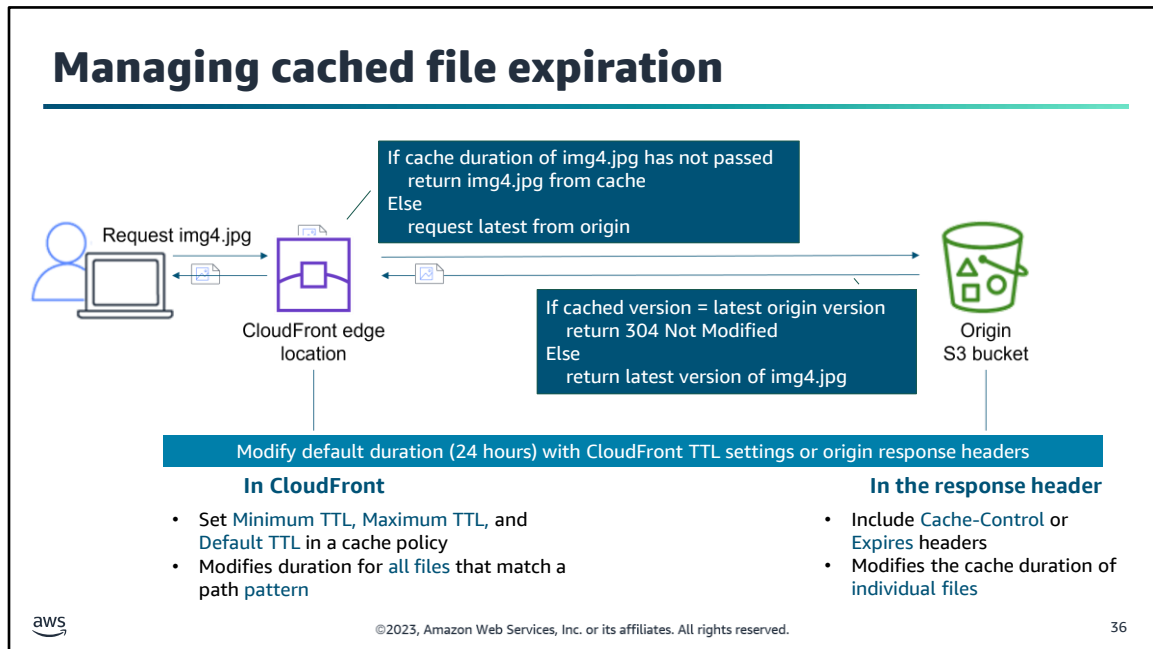
35

To control the cache key, you use a CloudFront *cache policy*. You attach a cache policy to one or more cache behaviors in a CloudFront distribution.

A cache policy contains settings that are categorized into *policy information*, *TTL settings*, and *cache key settings* as summarized in the table.

For more information about cache policy settings, see the Amazon CloudFront Developer Guide at <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/controlling-the-cache-key.html>.

CloudFront also provides managed cache policies so that you don't need to write or maintain your own cache policy. The managed policies use settings that are optimized for specific use cases. For more information about available managed cache policies, see the Amazon CloudFront Developer Guide at <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-managed-cache-policies.html>.



You can control how long your files stay in a CloudFront cache before CloudFront forwards another request to your origin.

- Reducing the duration means that you can serve dynamic content.
- Increasing the duration means that your users get better performance because your files are more likely to be served directly from the edge cache. A longer duration also reduces the load on your origin.

Typically, CloudFront serves a file from an edge location until the cache duration that you specified passes—that is, until the file expires. After it expires, the next time the edge location gets a user request for the file, CloudFront forwards the request to the origin server. In this way, it can verify that the cache contains the latest version of the file.

The response from the origin depends on whether the file has changed:

- If the CloudFront cache already has the latest version, the origin returns a status code 304 Not Modified.
- If the CloudFront cache does not have the latest version, the origin returns a status code 200 OK and the latest version of the file.

Each file in the cache automatically expires after 24 hours by default, but you can change the default behavior in two ways:

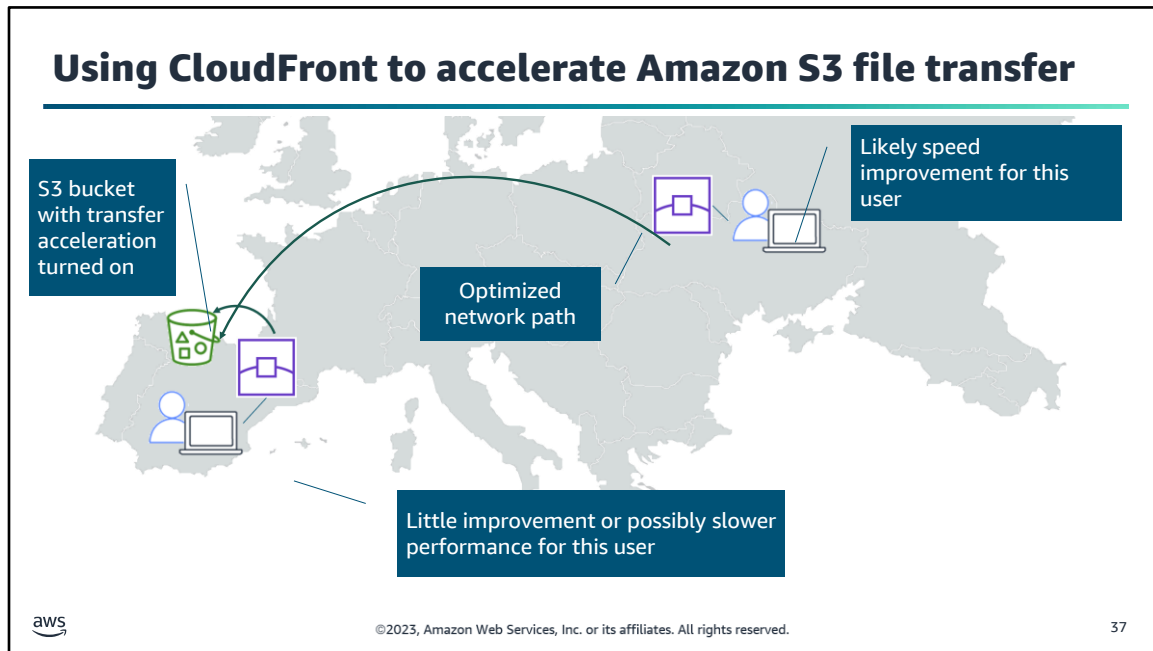
- You can change the cache duration for all files that match the same path pattern, as follows. Change the CloudFront settings for Minimum TTL, Maximum TTL, and Default TTL for a cache behavior by using a caching policy that is attached to the behavior.
- You can change the cache duration for an individual file, as follows. Configure your origin to add a Cache-Control max-age or Cache-Control s-maxage directive, or an Expires header field to the file. For more information about using headers to control cache duration for individual objects, see the Amazon CloudFront Developer Guide at

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Expiration.html>

on.html#expiration-individual-objects.

To control object caching, the Cache-Control max-age directive is recommended instead of the Expires header field. If you specify values both for Cache-Control max-age and for Expires, CloudFront uses only the value of Cache-Control max-age.

You can set these parameters on Amazon S3 objects directly from the Amazon S3 console. But when using CloudFront with Amazon S3 as an origin, you would usually set caching expiration behaviors in CloudFront rather than on the S3 objects. With CloudFront in front of your S3 bucket, you can configure Origin Identity Access (OIA) to prevent objects from ever being exposed except through CloudFront. Therefore, you want the CloudFront caching behaviors to control expiration of Amazon S3 objects. Be sure that you don't set up conflicting behaviors in CloudFront and on Amazon S3 objects.



Another way that CloudFront and Amazon S3 work together is the option to use Amazon S3 Transfer Acceleration. This bucket-level feature uses CloudFront's globally distributed edge locations to accelerate file transfers between a client and an S3 bucket over a long distance. As the data arrives at an edge location, data is routed to Amazon S3 over an optimized network path.

You might want to use Transfer Acceleration on a bucket for various reasons:

- Your customers upload to a centralized bucket from all over the world.
- You transfer gigabytes to terabytes of data on a regular basis across continents.
- You can't use all of your available bandwidth over the internet when uploading to Amazon S3.

This feature incurs an additional cost. However, depending on the locations of the client and the bucket, the file transfer might not be faster with this feature turned on. In general, the farther away the client is from an Amazon S3 Region, the greater is the speed improvement from using Amazon S3 Transfer Acceleration.

Consider the example that is illustrated. You'll likely see more benefit from using Amazon S3 Transfer Acceleration for the user who uploads files a long distance away from the S3 bucket. The user who is uploading files closer to the bucket might not see an improvement. They might actually see slower performance than they would get without the transfer acceleration feature turned on.

Test the performance on your selected buckets with access patterns that represent normal use.

Customizing at the edge with functions

- Use edge functions to customize how your distribution processes requests and responses.
- Code runs close to viewers to minimize latency.
- Two ways to write and manage edge functions:
 - Lambda@Edge
 - CloudFront Functions



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

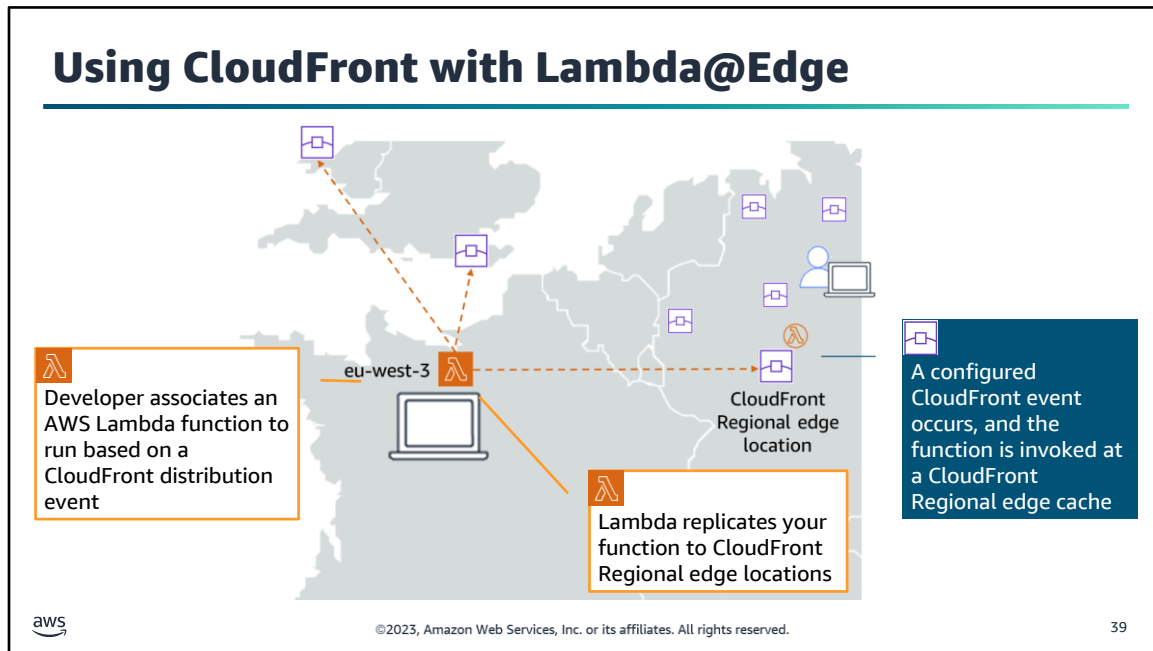
38

You can write your own code to customize how your CloudFront distributions process HTTP requests and responses. The code that you write and attach to your CloudFront distribution is called an *edge function*.

The code runs close to your viewers (users) to minimize latency, and you don't have to manage servers or other infrastructure. You can write code to manipulate the requests and responses that flow through CloudFront. You can also perform tasks like basic authentication and authorization or generation of HTTP responses at the edge.

CloudFront provides two ways to write and manage edge functions:

- **Lambda@Edge:** An extension of AWS Lambda that offers powerful and flexible computing for complex functions and full application logic closer to your viewers. It is also highly secure. Lambda@Edge functions run in a Node.js or Python runtime environment. You publish them to a single AWS Region. When you associate the function with a CloudFront distribution, Lambda@Edge automatically replicates your code around the world.
- **CloudFront Functions:** Lightweight JavaScript functions for high-scale, latency-sensitive CDN customizations. The CloudFront Functions runtime environment offers sub-millisecond startup times, scales immediately to handle millions of requests per second, and is highly secure. CloudFront Functions is a native feature of CloudFront, which means you can build, test, and deploy your code entirely within CloudFront.



This diagram illustrates how Lambda@Edge functions run. The Lambda boxes highlight steps that are managed by Lambda, while the CloudFront box highlights the steps managed by CloudFront. Lambda@Edge functions run in a Node.js or Python runtime environment. You publish them to a single AWS Region. When you associate the function with a CloudFront distribution, Lambda@Edge automatically replicates your code around the world.

Lambda@Edge functions are run in a Regional edge cache, usually in the AWS Region closest to the CloudFront edge location that the client can reach.

Lambda@Edge example

Lambda function code

```
def lambda_handler(event, context):
    request = event['Records'][0]['cf']['request']

    viewerCountry = request['headers'].get('cloudfront-viewer-country')
    if viewerCountry:
        countryCode = viewerCountry[0]['value']
        if countryCode == 'GB' or countryCode == 'DE' or countryCode == 'IE':
            domainName = 'eu.example.com'
            request['origin']['custom']['domainName'] = domainName
            request['headers']['host'] = [{'key': 'host', 'value': domainName}]
    return request
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

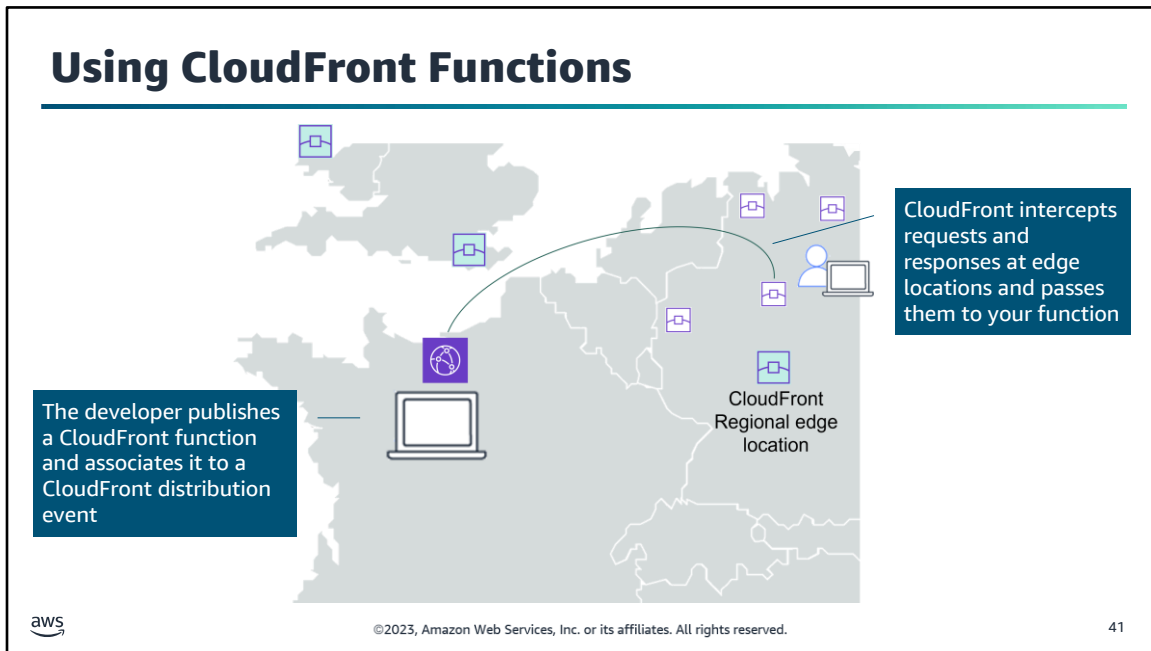
40

This Lambda function demonstrates how you can change the origin domain name based on the CloudFront-Viewer-Country header. In this way, content is served from an origin that is closer to the viewer's country.

Using this functionality for your distribution can have the following advantages:

- Reducing latencies when the specified Region is nearer to the viewer's country
- Providing data sovereignty by making sure that data is served from an origin that's in the same country that the request came from

To enable this functionality, you must configure your distribution to cache based on the CloudFront-Viewer-Country header.



Similar to Lambda@Edge, CloudFront Functions runs your code in response to events that CloudFront generates. CloudFront Functions can be invoked after CloudFront receives a request from a viewer (**viewer request**), before CloudFront forwards the response to the viewer (**viewer response**).

When you associate a CloudFront function with a CloudFront distribution, CloudFront intercepts requests and responses at CloudFront edge locations and passes them to your function.

CloudFront function example

CloudFront function code

```
function handler(event) {
  // NOTE: This example function is for
  // a viewer request event type.
  // Choose viewer request for event
  // type when you associate this function
  // with a distribution.
  var response = {
    statusCode: 302,
    statusDescription: 'Found',
    headers: {
      'cloudfront-functions': {
        value: 'generated-by-CloudFront-
        Functions' },
        'location': { value:
        'https://aws.amazon.com/cloudfront/' }
      }
    };
  return response;
}
```



CloudFront function redirects to:
<https://aws.amazon.com/cloudfront/>



Requested URL (the CloudFront distribution):
<https://d1111111abcdef8.cloudfront.net/>



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

42

This function code redirects the viewer to a different URL, and also returns a custom response header.

In this example, the developer sends a request to the distribution's domain name (<https://d1111111abcdef8.cloudfront.net/>).

The function redirects them to <https://aws.amazon.com/cloudfront> and adds the custom response header.

Choosing between Lambda@Edge and CloudFront Functions

	Lambda@Edge	CloudFront Functions
Use cases	Allows complex functions and full application logic	Lightweight , short-running functions
Ability to build and test entirely within CloudFront	No Extension of Lambda	Yes Native feature of CloudFront
Programming languages	Node.js or Python runtime	JavaScript
Event sources	Viewer request/response and origin request/response	Viewer request/response
Scale	Greater duration, memory capacity, and function size than CloudFront Functions	Limited to sub-milliseconds of duration, 2 MB memory, and 10 KB in size



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

43

Generally speaking, Lambda@Edge gives you more options and capacity for creating edge functions. However, CloudFront Functions gives you a quick way to handle lightweight functions at the edge at about one-sixth the cost of Lambda@Edge, all within CloudFront.

Lambda@Edge is a good choice for functions that have these characteristics:

- Take milliseconds or more to complete, or require adjustable memory or CPU
- Depend on third-party libraries, including the AWS SDKs, for integration with other AWS services
- Require network access to use external services for processing
- Require file system access or access to the body of HTTP requests

CloudFront Functions is a good choice for these example use cases:

- Cache key normalization
- Header manipulation
- Header URL redirects
- Request authorization

Section 4 key takeaways



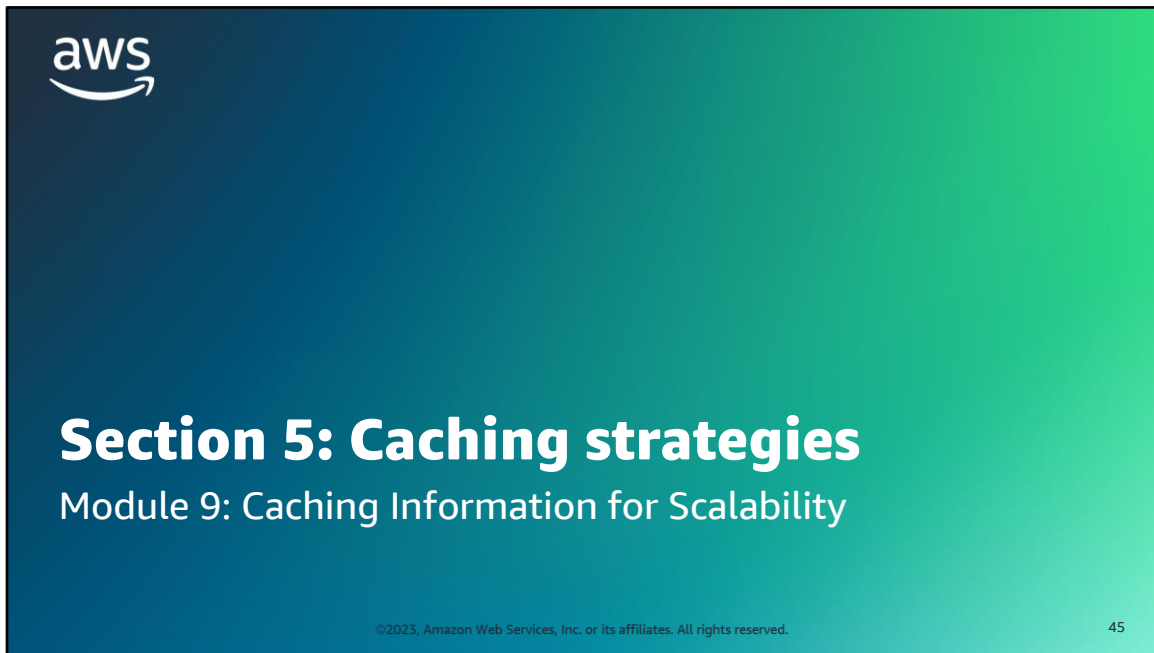
- CloudFront is a fast **CDN** service that delivers data globally with **low latency** and **high transfer speeds**.
- CloudFront distributions can serve **web**, **video**, or **live streaming** content.
- A **cache key** is the **unique identifier** for each object in the cache.
- You can use **CloudFront Functions** and **Lambda@Edge** to **customize** how your distribution processes requests and responses.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

44

The following are the key takeaways from this section of the module:

- CloudFront is a fast CDN service that securely delivers data globally with low latency and high transfer speeds.
- CloudFront distributions can serve web, video, or live streaming content.
- A cache key is the unique identifier for each object in the cache.
- You can use CloudFront Functions and Lambda@Edge to customize how your distribution processes requests and responses.



Section 5: Caching strategies.

Two primary caching strategies



Lazy loading

- The cache is updated only when necessary (cache hit or cache miss).
- Use when data will be read often but written infrequently.
- Example: User profile information



Write-through

- The cache is updated whenever data is written to the database.
- Use when data must be updated in real time.
- Examples: Top 100 game leaderboard or top 10 most popular news stories



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

46

Caching has two primary strategies: lazy loading and write-through.

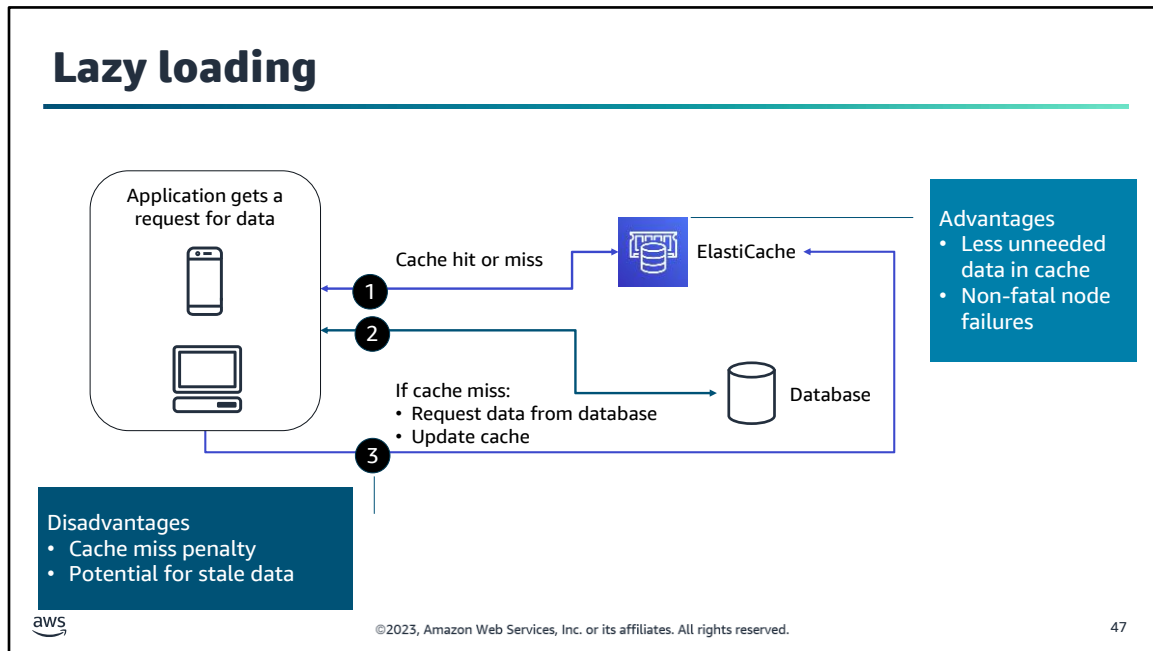
The lazy loading strategy updates the cache only when necessary, as in a *cache hit* or *cache miss* scenario. Use lazy loading when you have data that will be read often, but written infrequently.

For example, in a typical web or mobile application, a user's profile rarely changes, but it is accessed throughout the application. A person might update their profile only a few times per year. However, the profile might be accessed dozens or hundreds of times a day, depending on the user.

The write-through strategy adds data or updates data in the cache whenever data is written to the database. Use a write-through caching strategy when you have data that must be updated in real time. You can think of this strategy as being proactive: you can avoid unnecessary cache misses when you have data that you know will be accessed.

A good example is any type of aggregate, such as a top 100 game leaderboard, the top 10 most popular news stories, or even recommendations. Because a specific piece of application or background job code typically updates this data, it's straightforward to update the cache also.

The next slides use ElastiCache as an example to compare the two options.



With lazy loading, only requested data is cached.

An advantage is that lazy loading avoids filling up the cache with data that isn't requested. Additionally, node failures are not fatal. When a node fails, a new, empty node replaces it. The application continues to function, though with increased latency. As requests are made to the new node, each cache miss results in querying the database and adding the data copy to the cache. Subsequent requests for this data are retrieved from the cache.

The disadvantages include the cache miss penalty that was discussed earlier, and the potential to get stale data. If data is written to the cache only when you have a cache miss, then the cache isn't updated when data is changed in the database. Therefore, data in the cache can become stale.

Lazy loading example

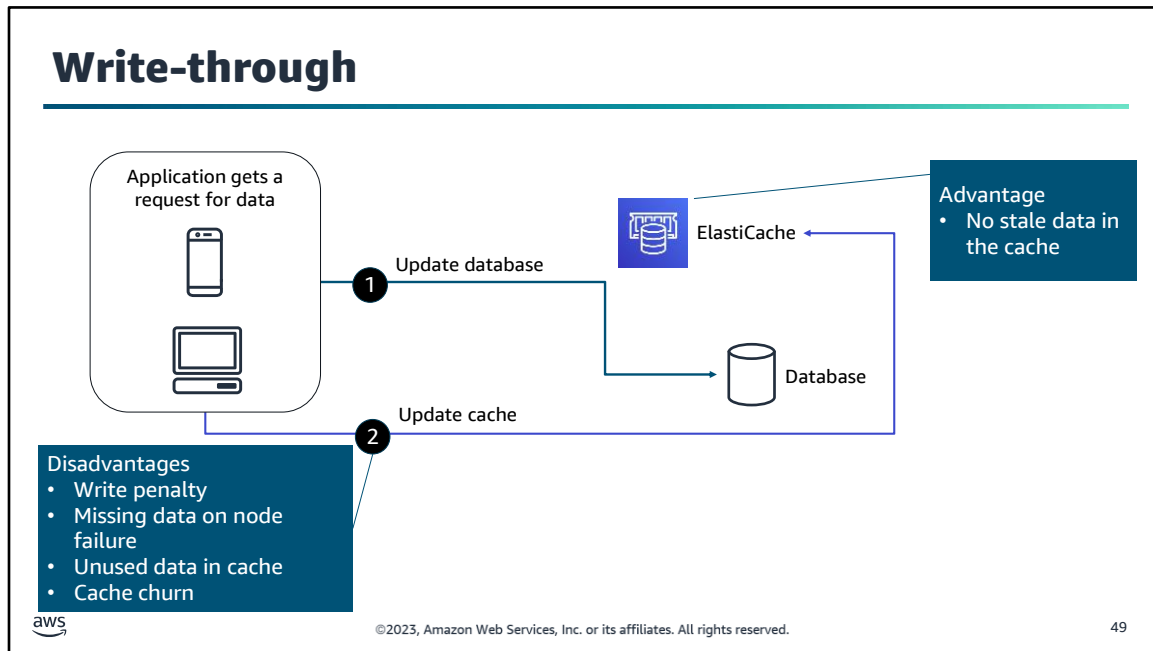
```
# Python
def get_user(user_id):
    # Check the cache
    record = cache.get(user_id)
    if record is None:
        # Run a DB query
        record = db.query("select * from users where id = ?", user_id)
        # Populate the cache
        cache.set(user_id, record)
    return record
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

48

This example shows the lazy loading workflow in Python pseudocode.



With write-through, when you update the database, you update the cache.

The advantage of this approach is that data in the cache is never stale. Because the data in the cache is updated every time it is written to the database, the data in the cache is always current.

The disadvantages of this approach include the following:

- **Write penalty:** Every write involves two trips—a write to the cache and a write to the database—which adds latency to the process. Still, users are generally more tolerant of latency when data is updated than when data is retrieved. Users have a general idea that updates are more work and thus might take longer.
- **Missing data:** When a new node is created to scale up or to replace a failed node, the node does not contain all of the data. Data continues to be missing until it is added or updated in the database. In this scenario, you might choose to use a lazy loading approach to repopulate the cache.
- **Unused data:** Since most data is never read, the cluster could have a large quantity of data that is never read.
- **Cache churn:** The cache might be updated often if certain records are updated repeatedly.

Write-through example

```
# Python
def save_user(user_id, values):
    # Save to DB
    record = db.query("update users ... where id = ?", user_id, values)
    # Push into cache
    cache.set(user_id, record)
    return record
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

50

This example shows the write-through workflow in Python pseudocode.

Section 5 key takeaways



- Caching has two primary strategies: **lazy loading** and **write-through**.
- With **lazy loading**, the cache is updated **only when necessary** (cache hit or miss).
- With **write-through**, the cache is updated **whenever data is written** to the database.
- Choose a caching strategy that **matches the use case** for your data.

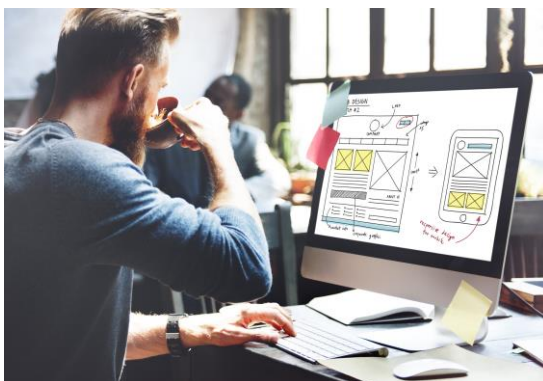
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

51

The following are the key takeaways from this section of the module:

- Caching has two primary strategies: lazy loading and write-through.
 - With lazy loading, the cache is updated only when necessary (cache hit or miss).
 - With write-through, the cache is updated whenever data is written to the database.
- Choose a caching strategy that matches the use case for your data.

Lab 9.2: Implementing CloudFront for Caching and Application Security



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

52

You will now complete Lab 9.2: Implementing CloudFront for Caching and Application Security.

Lab: Scenario 2

- The café website is still running on HTTP and does not yet support HTTPS.
- The café would like to ensure webpages will load quickly for users globally.
- Sofía decides it is time deliver the café website through a **content delivery network (CDN)**.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

53

Sofía is pleased how the café website development project is coming along. She has developed the core serverless application that displays menu items on the website, integrated the coffee supplier web application into the main site, and also leveraged the features of ElastiCache on the supplier side of the website. However, she knows that some essential features are still missing. One of them is that the website is still running on HTTP and does not yet support HTTPS. Also, she wants to ensure that the webpages will load quickly for users globally. She knows that although AWS has many Regions and Availability zones, AWS has even more Edge locations that are even closer to users around the globe. She decided that it is time to host the café website on a content delivery network (CDN).

Lab: Tasks 2

1. Preparing the development environment
2. Configuring an Amazon CloudFront distribution for static website content
3. Securing network access to the CloudFront distribution using AWS WAF
4. Securing a REST API endpoint using AWS WAF
5. Configuring a CloudFront Function on the website
6. Adjusting the cache duration

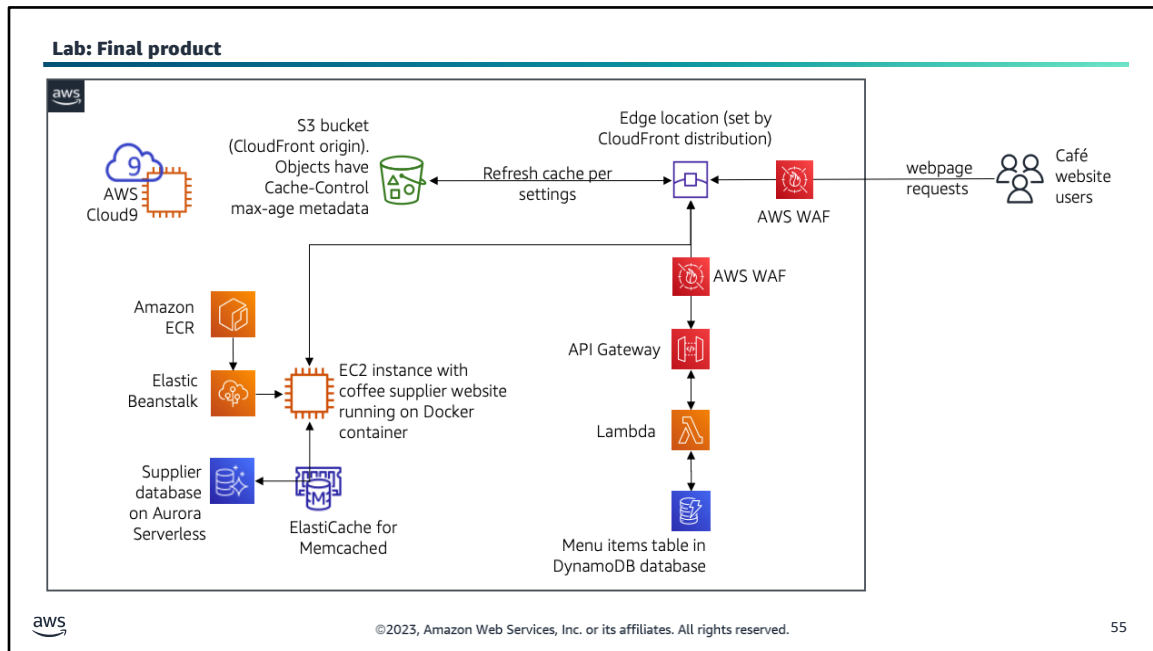


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

54

In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Configuring an Amazon CloudFront distribution for the static website content
3. Securing network access to the CloudFront distribution using AWS WAF
4. Securing a REST API endpoint using AWS WAF
5. Configuring a CloudFront Function on the website
6. Adjusting the cache duration



The diagram summarizes what will be configured in the AWS account after you complete the lab.

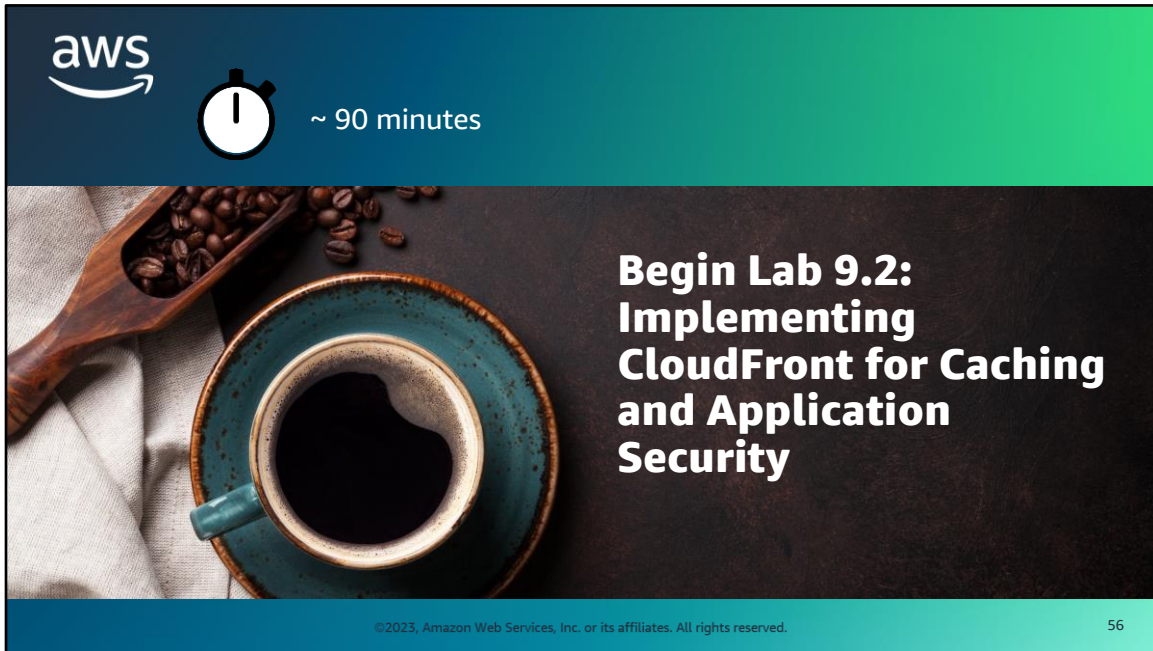
The diagram shows the following components of the deployed café website:

An Amazon ECR container image deployed through Elastic Beanstalk to an EC2 instance. The EC2 instance is running the coffee supplier website on a docker container.

An ElastiCache Memcached database that the coffee supplier website queries for cached data before connecting to the Aurora Serverless database.

End users accessing the café website connect through AWS WAF which controls access to the website resources. AWS WAF connects to an Amazon CloudFront distribution. The CloudFront distribution provides caching to the static website content stored in Amazon S3.

AWS WAF also secures access to an API Gateway endpoint that integrates with a Lambda function. The Lambda function connects to a DynamoDB database that stores menu items.



The image is a slide for an AWS lab. At the top, there is a dark blue header with the AWS logo on the left, a stopwatch icon in the center, and the text '~ 90 minutes' on the right. Below the header is a large image of a coffee cup on a saucer with coffee beans scattered around it. To the right of the coffee image, the text 'Begin Lab 9.2: Implementing CloudFront for Caching and Application Security' is displayed in white. At the bottom of the slide, there is a teal footer containing the copyright text '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the number '56' on the right.

aws ~ 90 minutes

**Begin Lab 9.2:
Implementing
CloudFront for Caching
and Application
Security**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 56

It is now time to start the lab.

Lab debrief: Key takeaways

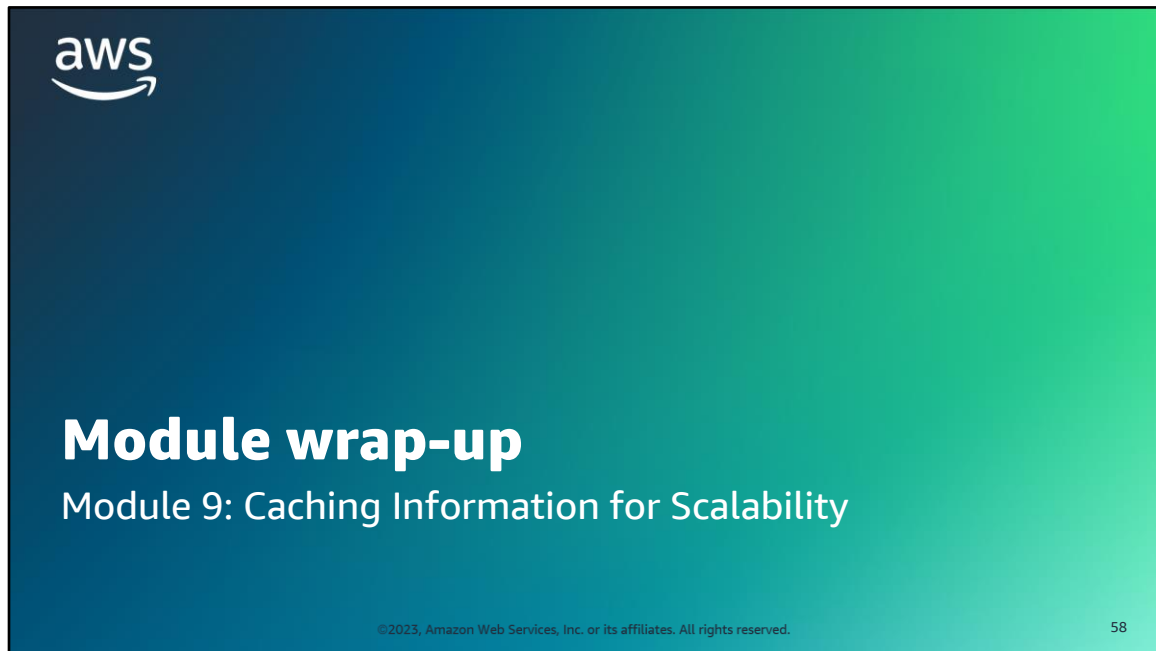


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

57

After you complete the lab, your educator might choose to lead a conversation about the key takeaways.



It's now time to review the module and wrap up with a knowledge check, and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to do the following:

- Explain when caching is used
- Describe caching with ElastiCache
- Describe caching with CloudFront
- Describe and compare Lambda@Edge and CloudFront Functions
- Apply caching strategies



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

59

In summary, in this module, you learned how to do the following:

- Explain when caching is used
- Describe caching with ElastiCache
- Describe caching with CloudFront
- Describe and compare Lambda@Edge and CloudFront Functions
- Apply caching strategies

Complete the knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

60

It is now time to complete the knowledge check for this module.

Sample exam question

Which requirement is a reason to choose Memcached over Redis for your ElastiCache configuration?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Requirement
- Memcached over Redis
- ElastiCache



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

61

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Responses

Which **requirement** is a reason to choose **Memcached over Redis** for your **ElastiCache** configuration?

Choice	Response
A	You need data replication.
B	You need the simplest model possible.
C	You have complex data types.
D	You need an in-memory cache.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 62

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is B.

Choice	Response
A	You need data replication.
B	You need the simplest model possible.
C	You have complex data types.
D	You need an in-memory cache.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 63

Look at the answer choices and rule them out based on the keywords that were previously highlighted.

The correct answer is B. You need the simplest model possible.

Both Redis and Memcached are in-memory caches. Use Memcached when you want to use the simplest model possible. Redis supports data replication and complex data types, but Memcached does not.

Additional resources

Video:

- [Getting Started with Caching: Turbocharging Your Application Workloads](#)

Blog post:

- [Introducing CloudFront Functions – Run Your Code at the Edge with Low Latency at Any Scale](#)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

64

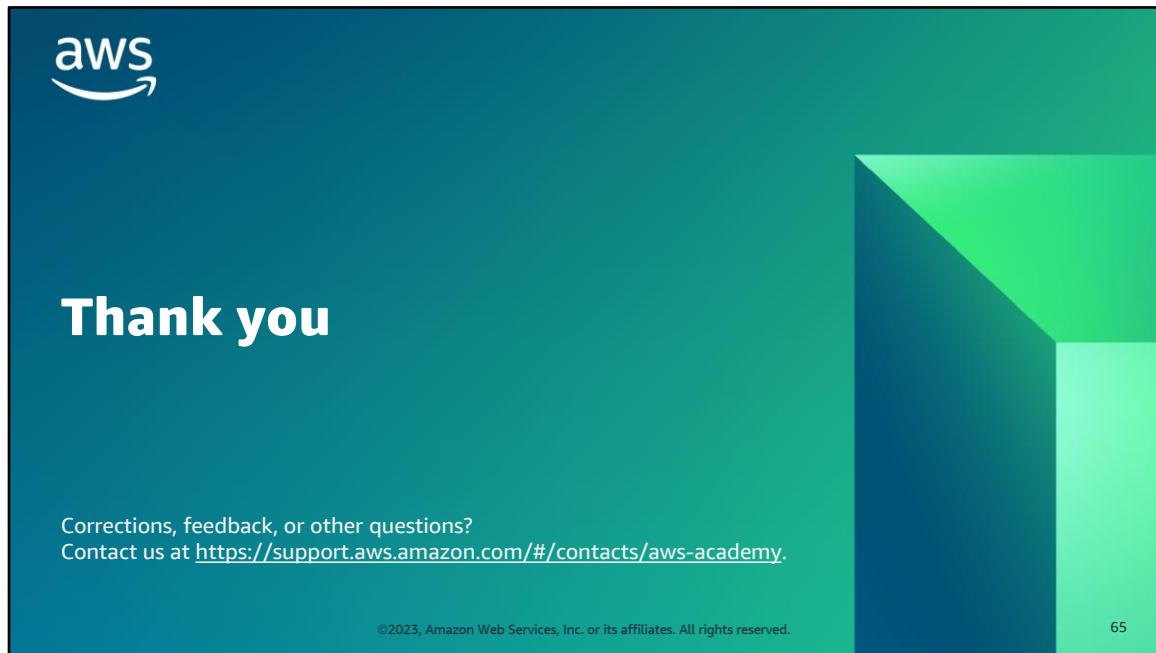
To learn more about the topics that are covered in this module, you might find the following additional resources helpful:

Videos:

- Getting Started with Caching: Turbocharging Your Application Workloads: https://pages.awscloud.com/Getting-Started-with-Caching-Turbocharging-Your-Application-Workloads_2021_0313-DAT_OD.html.

Blog post:

- Introducing CloudFront Functions – Run Your Code at the Edge with Low Latency at Any Scale: <https://aws.amazon.com/blogs/aws/introducing-cloudfront-functions-run-your-code-at-the-edge-with-low-latency-at-any-scale/>.



Thank you for completing this module.